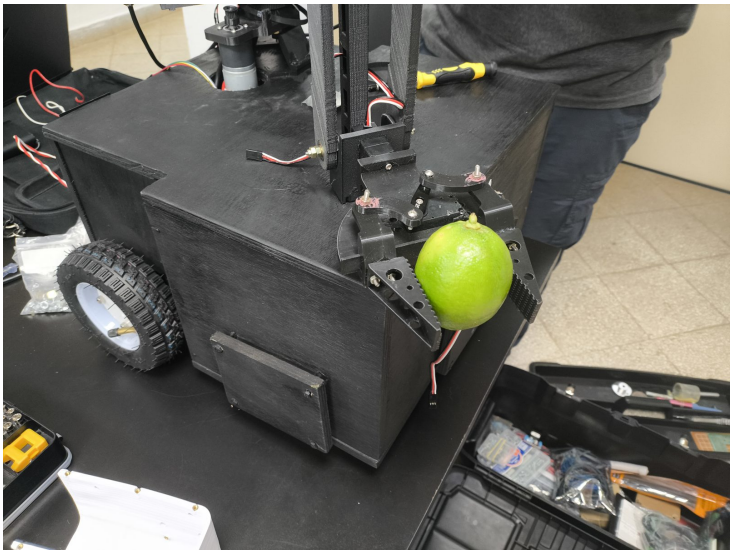
FACULTAD DE INGENIERÍA · UNIVERSIDAD NACIONAL DE ASUNCIÓN

Ingeniería Mecatrónica · Robótica I y Robótica II · 2026S2

# Arturito R2D2

## Robot móvil autónomo recolector de limones

Manual completo del sistema: hardware, electrónica, firmware, ROS 2, visión y operación






**Grupo 2**

Versión 1.0 · 14 de octubre de 2026 · estado del proyecto a esa fecha

Repositorios: `HecVelaz/recolector_de_frutas` (robot) y `HecVelaz/vision_estereo` (visión)

# Índice general

**1. Introducción** **1**

1.1. Qué es Arturito R2D2 . . . 1

1.2. Cómo leer este manual . . . 1

1.3. Glosario rápido . . . 2

**2. El sistema completo en una mirada** **4**

2.1. La idea central: la Pi piensa, los ESP32 ejecutan . . . 4

2.2. Diagrama de bloques . . . 5

2.3. Dónde está cada cosa en el robot . . . 6

2.4. Un limón, de principio a fin . . . 6

2.5. Las capas de seguridad . . . 8

**3. Mecánica: base, ruedas, brazo y pinza** **9**

3.1. La base . . . 9

3.2. Tracción diferencial . . . 9

3.2.1. Cómo se mueve un robot diferencial . . . 9

3.2.2. Ruedas y motores de tracción . . . 10

3.3. El brazo . . . 11

3.3.1. Tipo y geometría . . . 11

3.3.2. Dónde va montado y qué alcanza . . . 12

3.3.3. Motores y transmisiones . . . 12

3.4. La pinza . . . 13

**4. Alimentación y potencia** **14**

4.1. Dos fuentes separadas . . . 14

4.2. El circuito, paso a paso . . . 14

4.3. Cada pieza . . . 15

4.4. Consumo y autonomía . . . 16

4.5. La batería sin BMS . . . 17

4.6. Reglas para las pruebas . . . 17

**5. Placas, pines y cableado** **18**

5.1. Vista general del cableado . . . 18

5.2. Tres ideas eléctricas que aparecen en todas partes . . . 18

5.2.1. Los GPIO del ESP32 son de 3,3 V . . . . . 18
5.2.2. Pull-up y pull-down . . . . . 19
5.2.3. La seta es normalmente cerrada (falla segura) . . . . . 19
5.3. ESP32-S3: la base . . . . . 19
5.4. ESP32-WROOM-32D: el brazo . . . . . 21
5.5. Raspberry Pi 5 . . . . . 23
5.6. La seta, cable por cable . . . . . 24

**6. Firmware de los ESP32** **25**
6.1. Organización del código . . . . . 25
6.1.1. Entornos de compilación . . . . . 25
6.2. Cómo corre el programa . . . . . 26
6.3. El control de un motor: cascada posición → velocidad . . . . . 26
6.3.1. Parámetros de J1 (medidos el 02/10) . . . . . 27
6.4. Firmware de la base (S3) . . . . . 27
6.4.1. Velocidad de las ruedas . . . . . 27
6.4.2. Odometría . . . . . 27
6.4.3. Seguridad de la base . . . . . 27
6.4.4. Comandos por el monitor serie (base) . . . . . 28
6.5. Firmware del brazo (WROOM) . . . . . 29
6.5.1. El cero y el homing . . . . . 29
6.5.2. Protecciones del brazo . . . . . 29
6.5.3. Comandos por el monitor serie (brazo) . . . . . 29

**7. ROS 2 desde cero** **31**
7.1. Los conceptos . . . . . 31
7.1.1. Nodos, tópicos y mensajes . . . . . 31
7.1.2. Otras piezas . . . . . 31
7.2. Preparar una terminal . . . . . 32
7.3. Mirar qué está pasando . . . . . 32
7.4. Publicar a mano: `ros2 topic pub` . . . . . 33
7.4.1. Ejemplos con el robot . . . . . 33
7.5. Los tipos de mensaje que usa el robot . . . . . 34
7.6. Escribir un nodo propio . . . . . 34
7.7. Un *launch* mínimo . . . . . 35
7.8. Marcos de coordenadas (TF) del robot . . . . . 36

**8. micro-ROS: cómo hablan la Pi y los ESP32** **37**

8.1. Qué es micro-ROS . . . 37
8.1.1. Arrancar los agentes (en la Pi) . . . 37
8.2. La sesión: conexión, hora y ping . . . 37
8.3. Armar y desarmar . . . 38
8.3.1. Por qué existe . . . 38
8.3.2. Quién arma . . . 38
8.3.3. Armar no es todo . . . 38
8.4. Watchdog de comandos . . . 39
8.5. Tópicos de la base . . . 39
8.6. Tópicos del brazo . . . 39

**9. Visión: encontrar el limón en 3D** **41**
9.1. La cadena completa . . . 41
9.2. Las cámaras . . . 41
9.3. Cómo funciona el estéreo . . . 41
9.3.1. La idea . . . 41
9.3.2. Calibración y rectificación . . . 42
9.3.3. Disparidad: SGBM . . . 42
9.4. El detector YOLO . . . 43
9.5. El localizador: de la caja al punto 3D . . . 43
9.6. Resultados medidos . . . 44
9.7. Cómo se usa . . . 44

**10. Cinemática del brazo** **46**
10.1. El problema . . . 46
10.2. Cinemática directa . . . 46
10.3. Cinemática inversa, paso a paso . . . 46
10.4. Del punto de agarre a la muñeca . . . 47
10.5. El nodo `brazo_ik` . . . 47

**11. Misión y recorrido autónomo** **49**
11.1. La misión (brazo) . . . 49
11.1.1. Cada estado . . . 49
11.1.2. Qué significa “llegó” . . . 50
11.1.3. La seta y la misión . . . 50
11.2. El recorrido (base) . . . 50
11.2.1. Cuándo frena por un limón . . . 51
11.2.2. Cuánto avanzar para alcanzar el limón . . . 51

11.2.3. Rumbo: ruedas para avanzar, giróscopo para girar . . . 52
11.2.4. Esquivar . . . 52
11.3. Cómo se lanza . . . 52

**12.Simulación en Gazebo** **54**
12.1. Qué hay en el mundo . . . 54
12.2. Quién imita a quién . . . 54
12.3. Cómo se usa . . . 54
12.4. Resultados . . . 55

**13.Puesta en marcha, paso a paso** **56**
13.1. Paso 0: antes de conectar . . . 56
13.2. Paso 1: encendido y agentes . . . 57
13.3. Paso 2: base con las ruedas en el aire . . . 57
13.4. Paso 3: base en el piso . . . 57
13.5. Paso 4: sensores en ROS . . . 58
13.6. Paso 5: visión . . . 58
13.7. Paso 6: brazo con J1 . . . 58
13.8. Paso 7: J2 y J3 (cuando lleguen los motores) . . . 58
13.9. Paso 8: TF cámara → brazo . . . 59
13.10Paso 9: misión . . . 59
13.11Paso 10: recorrido autónomo . . . 59

**14.Diagnóstico: qué hacer si algo no anda** **60**
14.1. Primero, mirar el estado . . . 60
14.2. Tabla de síntomas . . . 60
14.3. Comandos útiles de la Pi . . . 61

**15.Estado del proyecto y lo que falta** **63**
15.1. Qué está hecho y qué no (14/10/2026) . . . 63
15.2. Valores marcados MEDIR . . . 63
15.3. Plan hasta la entrega (9/11/2026) . . . 64
15.4. Mantener la Pi al día . . . 64

**A. Referencia rápida** **65**
A.1. Todos los tópicos . . . 65
A.2. Comandos de todos los días . . . 65
A.3. Estructura del repositorio . . . 66
A.4. Documentos relacionados . . . 66

CAPÍTULO 1

# Introducción

## 1.1 Qué es Arturito R2D2

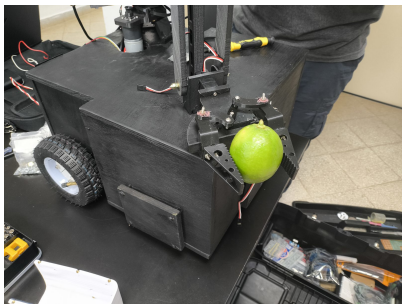
Arturito R2D2 es un robot móvil que recorre el pasillo entre dos hileras de limoneros, encuentra los limones con una cámara, calcula dónde están en el espacio y los cosecha con un brazo articulado. Es el trabajo práctico final del Grupo 2 para dos materias a la vez:

- **Robótica I** evalúa el **brazo**: un manipulador RRR (tres articulaciones rotativas) tipo uArm / Dobot Magician, con una pinza de dos servos.
- **Robótica II** evalúa el **robot móvil**: la base con tracción diferencial, los sensores, la seguridad, la integración con ROS 2 y la autonomía.

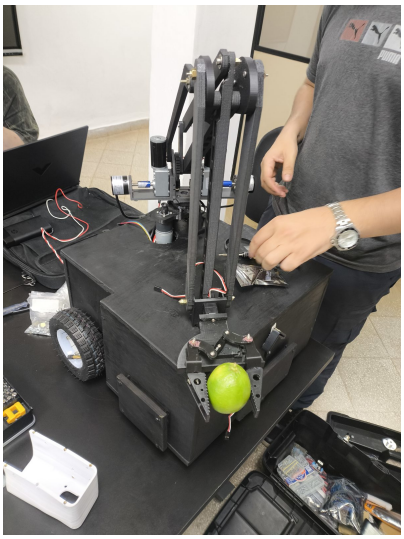
**Objetivo mínimo para aprobar**

El robot **se mueve**, **detecta el limón con YOLO**, **lo localiza en 3D con visión estéreo** y **lo agarra** con el brazo. Todo lo demás (GPS, app móvil, clasificación por tamaño, 30 frutos medidos) viene después de que esto funcione de punta a punta.

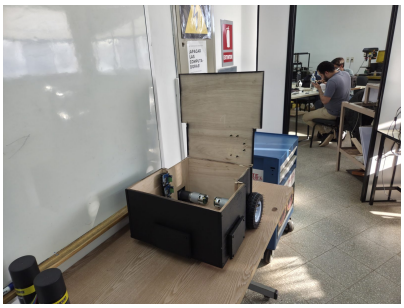
El limón no figura entre las especies de la guía de Robótica II, pero su uso fue aprobado por el profesor. También se aprobó que la herramienta sea una **pinza con agarre, tracción y giro** en vez de la ventosa de succión que proponía la consigna.



**(a)** Agarrando un limón.



**(b)** Brazo sobre la base.



**(c)** Base de madera en T.

**Figura 1.1:** El robot en el laboratorio (fotos de las entregas R1 E6 y R2 E3).

## 1.2 Cómo leer este manual

El manual va de lo físico a lo lógico y termina con procedimientos. Se puede leer de corrido o ir directo a lo que se necesite:

| Si querés... | Leé |
|---|---|
| Entender el sistema completo en 10 minutos | Capítulo 2 (vista general) |
| Saber qué es cada pieza física | Capítulos 3 (mecánica) y 4 (potencia) |
| Cablear una placa | Capítulo 5 |
| Entender qué hace el código de los ESP32 | Capítulo 6 |
| Aprender ROS 2 desde cero (publicar, suscribir, *launch*) | Capítulo 7 |
| Entender el "armar" y cómo hablan la Pi y los ESP32 | Capítulo 8 |
| Entender cómo se encuentra el limón en 3D | Capítulo 9 |
| Entender cómo se calculan los ángulos del brazo | Capítulo 10 |
| Entender cómo decide el robot cuándo frenar y cuánto avanzar | Capítulo 11 |
| Probar todo sin hardware | Capítulo 12 (Gazebo) |
| Encender el robot por primera vez, paso a paso | Capítulo 13 |
| Arreglar algo que no anda | Capítulo 14 |
| Saber qué falta | Capítulo 15 |
| Buscar un tópico, un comando o un parámetro | Apéndice A |

### Convenciones

- `/cmd_vel`: un **tópico** de ROS 2 (un "canal" de mensajes).
- `firmware/base/include/pins.h`: un **archivo** del repositorio `recolector_de_frutas`.
- `armar`: un **comando** o texto que se escribe tal cual.
- Unidades del Sistema Internacional en todo ROS 2: metros, radianes, segundos. El firmware del brazo trabaja internamente en grados y la capa micro-ROS convierte.
- Los valores marcados **MEDIR** son estimaciones: hay que reemplazarlos por lo medido en el robot real.

**Cajas de este manual**

Las cajas azules son notas; las verdes, resúmenes; las rojas, **cosas que pueden romper algo o lastimar**; las naranjas, valores o pruebas que todavía faltan.

## 1.3 Glosario rápido

| Término | Qué significa acá |
|---|---|
| ROS 2 (Jazzy) | *Robot Operating System*: no es un sistema operativo sino un conjunto de bibliotecas y herramientas para que muchos programas (nodos) intercambien mensajes. Corre en la Raspberry Pi. |
| Nodo | Un programa de ROS 2. Ej.: el detector YOLO, la cinemática inversa, la misión. |
| Tópico | Canal con nombre por donde viajan mensajes de un tipo fijo. Ej.: `/cmd_vel` lleva `geometry_msgs/Twist`. |
| Publicar / suscribir | Mandar mensajes a un tópico / recibirlos. Cualquiera puede publicar o suscribirse. |
| micro-ROS | ROS 2 reducido que corre en un microcontrolador (los ESP32). Necesita un **agente** en la Pi que lo conecte con el resto. |

| **Término** | **Qué significa acá** |
|---|---|
| Agente micro-ROS | Programa en la Pi que hace de puente entre el ESP32 (por USB serie) y la red de ROS 2. |
| Armar / desarmar | Seguro por sesión de cada ESP32: desarmado no acepta órdenes de movimiento (capítulo 8). |
| ESP32-S3 | Microcontrolador de la **base** (tracción, sensores, seguridad). |
| ESP32-WROOM-32D | Microcontrolador del **brazo** (J1, J2, J3 y la pinza). |
| IBT-2 (BTS7960) | Puente H de potencia que mueve un motor DC en los dos sentidos. Hay cinco. |
| PWM | Modulación por ancho de pulso: cómo el ESP32 le dice al IBT-2 "cuánta tensión" darle al motor. |
| Encoder | Sensor en el eje del motor que cuenta pulsos: dice cuánto y a qué velocidad giró. |
| PCNT | Contador de pulsos por hardware del ESP32: cuenta los pulsos del encoder sin usar la CPU. |
| PID / PI | Controlador que corrige la salida según el error entre lo pedido y lo medido. |
| Odometría | Estimar cuánto avanzó y giró el robot integrando lo que midieron los encoders de las ruedas. |
| IMU | Unidad inercial (MPU-6500): acelerómetro y giróscopo. Da el rumbo y detecta vuelcos. |
| HC-SR04 | Sensor de distancia por ultrasonido (hay tres al frente). |
| Bumper | Microinterruptor que se aprieta al chocar (hay tres: frente e izquierda y derecha). |
| Seta | Botón rojo de parada de emergencia. Corta los 12 V de los motores. |
| Estéreo | Dos cámaras separadas 9 cm: comparando las dos imágenes se calcula la distancia. |
| Disparidad | Cuántos píxeles se corre un punto entre la imagen izquierda y la derecha. Más disparidad = más cerca. |
| YOLO | Red neuronal que encuentra objetos (limones) en una imagen y les dibuja una caja. |
| TF | Sistema de ROS 2 que guarda dónde está cada marco de coordenadas (cámara, brazo, base) respecto de los otros. |
| TCP | *Tool Center Point*: el punto de agarre, entre los dedos de la pinza. |
| IK | Cinemática inversa: dado un punto (x, y, z), calcular los ángulos de J1, J2 y J3. |
| FSM | Máquina de estados finitos: el "cerebro" de la misión (ESPERAR, APROXIMAR, AGARRAR…). |
| Watchdog | Temporizador que frena todo si deja de llegar una señal periódica. |
| LiPo 3S | Batería de polímero de litio de 3 celdas en serie (11,1 V nominal). |
| Buck | Convertidor que baja la tensión (12 V a 6 V para los servos). |

CAPÍTULO 2

# El sistema completo en una mirada

## 2.1 La idea central: la Pi piensa, los ESP32 ejecutan

El robot tiene **tres computadoras**, cada una con un trabajo distinto:

**Raspberry Pi 5** (Ubuntu 24.04, ROS 2 Jazzy). Es el "cerebro": ve con las cámaras, detecta limones con YOLO, calcula su posición 3D, decide qué hacer (misión y recorrido) y calcula los ángulos del brazo. No maneja ningún motor directamente: solo **manda consignas**.

**ESP32-S3 (base)** Mueve las dos ruedas de tracción a la velocidad pedida, cuenta los pulsos de los encoders, calcula la odometría, lee la IMU, los 3 ultrasonidos, los 3 bumpers, la seta y la batería, y **decide sola la seguridad de la base**: si algo anda mal, frena sin preguntarle a la Pi.

**ESP32-WROOM-32D (brazo)** Lleva J1, J2 y J3 a los ángulos pedidos con un control en cascada y maneja los dos servos de la pinza. También se frena sola ante fallas (atasco, límites, encoder roto, pérdida de la Pi).

**En pocas palabras**

La Pi dice **qué** hacer ("andá a 0,1 m/s", "poné J2 a 45°"). Cada ESP32 decide **cómo** hacerlo (cuánto PWM darle a cada motor, 100 veces por segundo) y **si es seguro** hacerlo. Si la Pi se cuelga o se desconecta el USB, cada ESP32 se detiene sola en menos de medio segundo.

Hablan entre sí por **USB** usando **micro-ROS**: para el resto de ROS 2, cada ESP32 aparece como un nodo más que publica y se suscribe a tópicos.

## 2.2 Diagrama de bloques

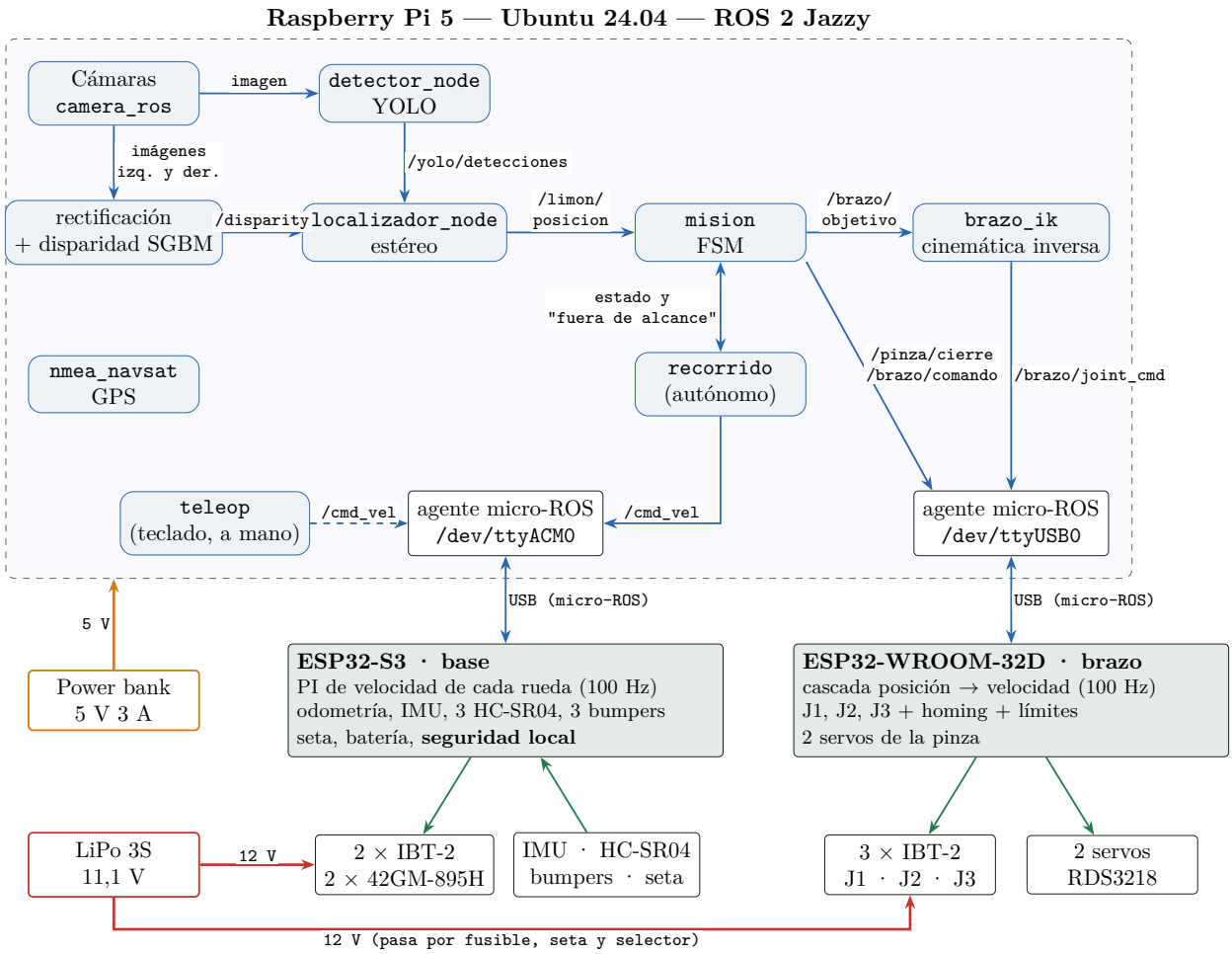



**Figura 2.1:** Arquitectura del sistema. Azul: nodos de ROS 2 en la Pi. Gris: placas. Las flechas llevan el nombre del tópico de ROS 2 por el que viaja el dato. `recorrido` también escucha `/yolo/detecciones`, `/odom`, `/range/*`, `/bumpers` y `/imu/data_raw` (no dibujados). El teclado (`teleop`) se usa en vez del recorrido autónomo, nunca a la vez.

## 2.3 Dónde está cada cosa en el robot

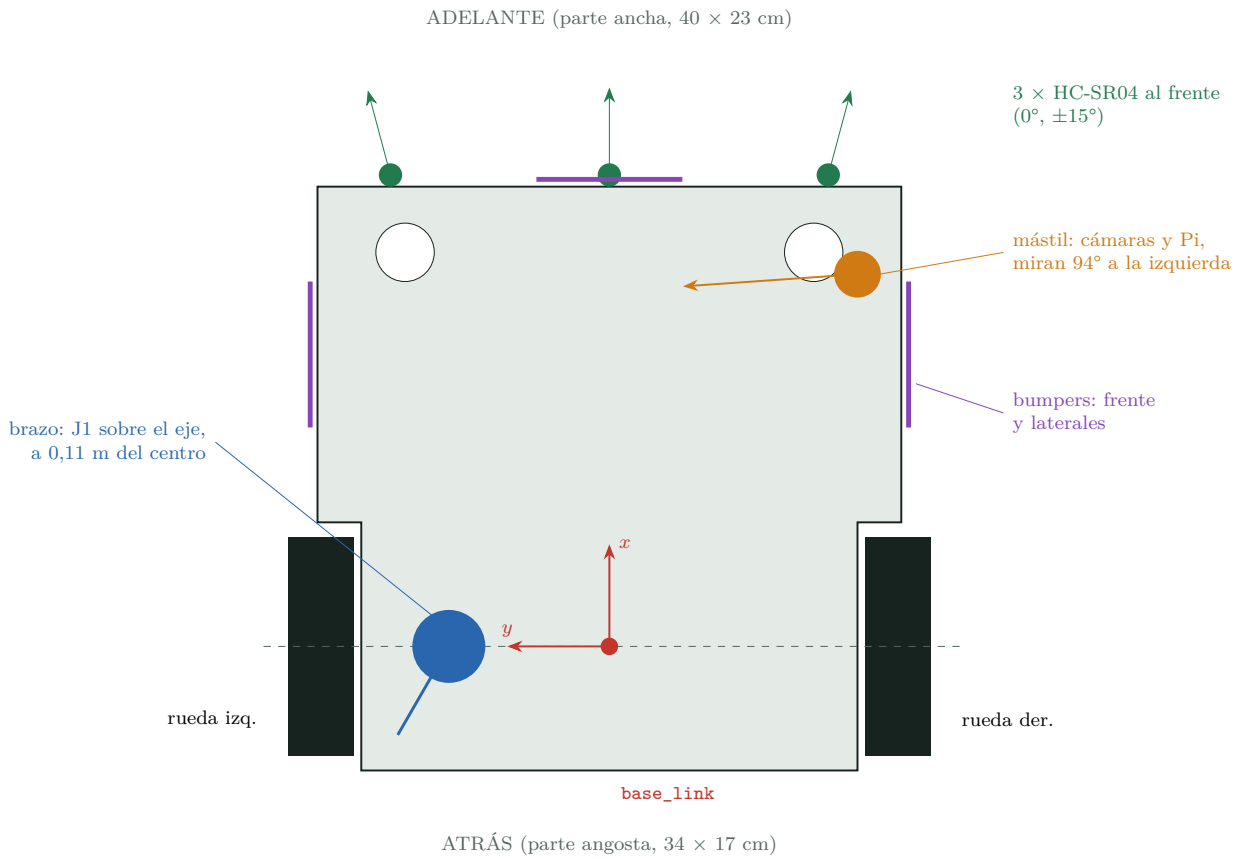



**Figura 2.2:** Vista desde arriba (medidas de la entrega R2 E3 y de la simulación; posiciones del mástil y del brazo **preliminares**). El origen `base_link` está en el eje de las ruedas de tracción, a 8,5 cm del borde trasero. Los limones están a la **izquierda** del robot: el brazo y las cámaras miran hacia ese lado.

## 2.4 Un limón, de principio a fin

Esta es la secuencia completa de una cosecha autónoma. Cada paso se explica en detalle más adelante.

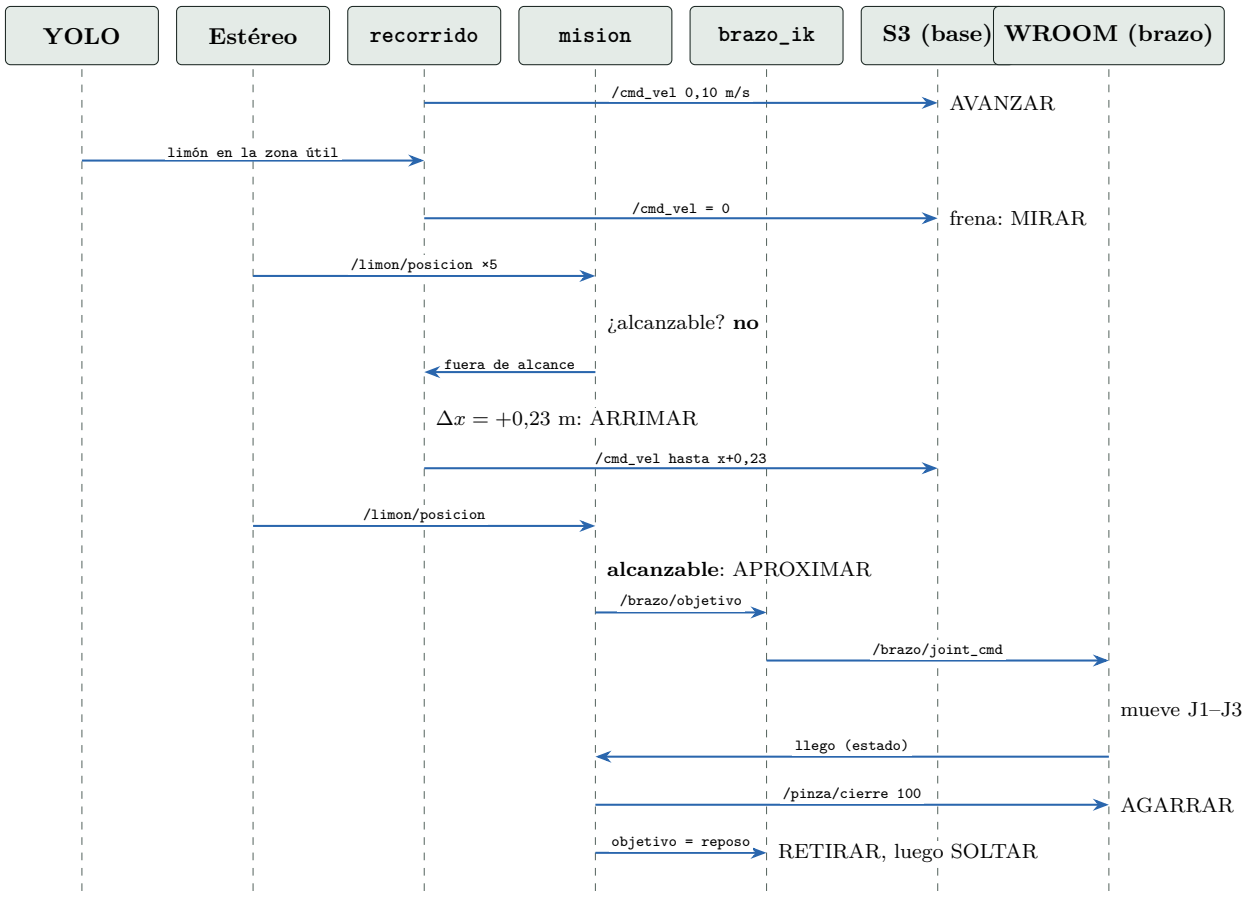



**Figura 2.3:** Diagrama de secuencia de una cosecha con el recorrido autónomo. La base **nunca** se mueve mientras el brazo trabaja.

En palabras:

1. **AVANZAR**: `recorrido` manda `/cmd_vel` con 0,10 m/s por el centro del pasillo. La S3 convierte eso en rpm para cada rueda y las controla con sus encoders.
2. YOLO ve un limón. Con el ancho de la caja se estima si está a una distancia que el estéreo puede medir (0,38–0,96 m). Si es así, `recorrido` **frena**: el estéreo necesita el robot quieto porque las dos cámaras no toman la foto exactamente al mismo tiempo.
3. Quieto, el estéreo publica la posición 3D del limón en `/limon/posicion`. `mision` junta 5 muestras que coincidan a menos de 2 cm, pasa el punto al marco del brazo con la TF y pregunta a la cinemática inversa si es alcanzable.
4. Si **no** es alcanzable, `recorrido` calcula cuántos centímetros avanzar o retroceder por el pasillo para que lo sea, y se mueve esa distancia midiendo con la odometría.
5. Si es alcanzable, `mision` manda el punto a `brazo_ik`, que calcula los ángulos de las tres articulaciones y se los manda a la WROOM.
6. La WROOM mueve J1, J2 y J3. Cuando las tres llegaron, `mision` cierra la pinza, lleva el brazo a la pose de reposo y abre la pinza para soltar el limón.
7. Vuelve a ESPERAR y `recorrido` sigue avanzando hasta el próximo limón o el fin de la hilera.

## 2.5 Las capas de seguridad

**1. Hardware**: la seta corta el + de los 12 V de todos los motores. Funciona aunque se cuelgue todo el software. El fusible de 20 A protege la LiPo de un cortocircuito.

**2. Firmware de cada ESP32**: watchdog de 300 ms, bumpers, distancia al frente, vuelco, atasco de motores, límites articulares, encoder roto. Decide sin la Pi.

**3. Armado por sesión**: después de cada conexión con la Pi, los ESP32 ignoran las órdenes de movimiento hasta recibir `armar`.

**4. Misión en la Pi**: si ve `SETA` en `/base/estado`, desarma el brazo y pasa a FALLA; la base solo se mueve con el brazo en reposo; si el brazo no llega en 20 s, FALLA.

**5. Operador**: pruebas acotadas y vigiladas, con alguien con la mano en el selector o la seta.

**Figura 2.4:** Capas de seguridad, de la más dura (hardware) a la más blanda (procedimientos).

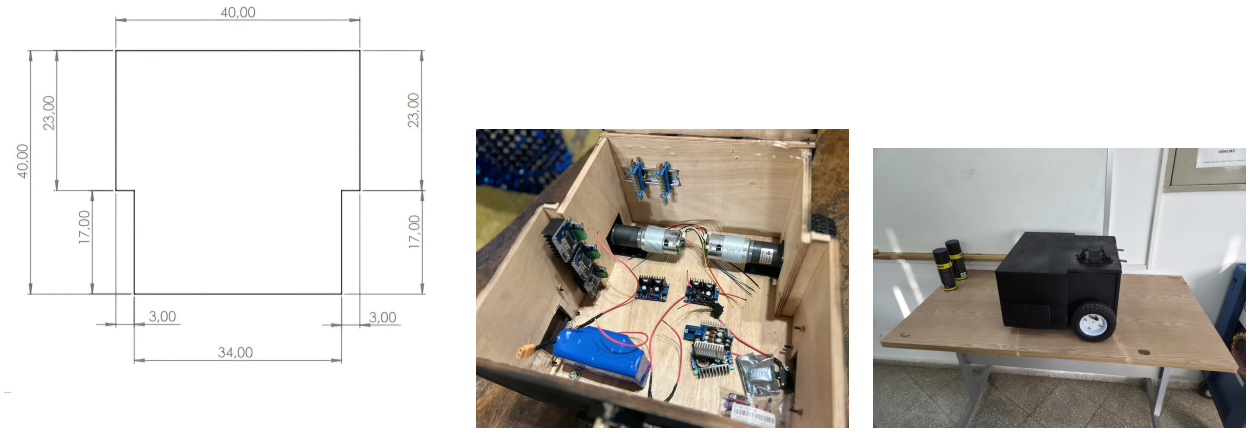
CAPÍTULO 3

# Mecánica: base, ruedas, brazo y pinza

## 3.1 La base

La base es una **caja de madera de 1 cm en forma de T**, de 40 cm de largo y unos 20 cm de alto (paredes de 18 cm), pintada de negro. Se definió en la entrega R2 E3.

- **Adelante, la parte ancha**: 40 × 23 cm. Debajo van las dos ruedas locas.
- **Atrás, la parte angosta**: 34 × 17 cm, con las dos ruedas de tracción en rebajes de 3 cm a cada lado. Así las ruedas no sobresalen del ancho de la parte delantera.
- El origen de coordenadas del robot, `base_link`, está en el **eje de las ruedas de tracción**, a 8,5 cm del borde trasero (figura 2.2). $x$ apunta hacia adelante, $y$ a la izquierda y $z$ hacia arriba (convención REP-103 de ROS).
- Peso estimado del robot completo: 6–9 kg (la guía pide $\geq 5$ kg). **MEDIR**.



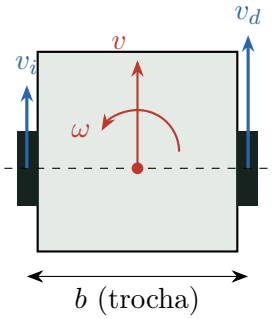

**(a)** Planta de la tapa (cm). **(b)** Interior con la batería vieja. **(c)** Vista lateral.

**Figura 3.1:** La base de madera (entrega R2 E3).

## 3.2 Tracción diferencial

### 3.2.1 Cómo se mueve un robot diferencial

El robot tiene **dos ruedas motrices independientes** sobre el mismo eje y dos ruedas locas que solo apoyan. No tiene dirección: para girar, una rueda va más rápido que la otra.




$$v = \frac{v_d + v_i}{2} \qquad \omega = \frac{v_d - v_i}{b}$$

$$v_i = v - \omega\,\frac{b}{2} \qquad v_d = v + \omega\,\frac{b}{2}$$

$$\mathrm{rpm}_{i,d} = \frac{60\,v_{i,d}}{2\pi r}$$

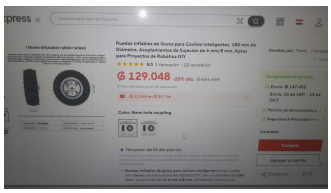
**Figura 3.2:** Cinemática diferencial. $v$ es la velocidad de avance del centro del eje [m/s], $\omega$ la velocidad de giro [rad/s, positiva a la izquierda], $r = 0{,}075$ m el radio de rueda y $b = 0{,}40$ m la trocha (estimada: **MEDIR**).

Ejemplos:

- $v = 0{,}10$ m/s, $\omega = 0$: las dos ruedas a $\frac{60 \cdot 0{,}10}{2\pi \cdot 0{,}075} \approx 12{,}7$ rpm. Avanza recto.
- $v = 0$, $\omega = 0{,}5$ rad/s: $v_d = +0{,}10$, $v_i = -0{,}10$ m/s. Gira sobre su eje hacia la izquierda.
- $v = 0{,}10$, $\omega = 0{,}25$: $v_d = 0{,}15$, $v_i = 0{,}05$ m/s. Avanza curvando a la izquierda con radio $v/\omega = 0{,}4$ m.

Esto es lo que hace la clase `Diferencial` del firmware de la base (`firmware/base/lib/BaseLogica/`): recibe $(v, \omega)$ de `/cmd_vel`, calcula las rpm de cada rueda y, si alguna supera el máximo, **reduce las dos en la misma proporción** para conservar la curvatura (el robot hace la misma curva, más despacio). Además aplica una **rampa**: la velocidad cambia como mucho 0,3 m/s² y 1,5 rad/s², para que el robot no pegue tirones.

### 3.2.2 Ruedas y motores de tracción



**(a)** Rueda inflable de 150 mm.



**(b)** 42GM-895H con encoder.

Hall encoder 42GM-895H planetary gear DC reduction motor

(Input speed 10000rpm)

| | | | 3.7 | 14 | 19 | 50 | 71 | 100 | 189 | 262 | 369 | 516 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Voltage | Reduction Ratio | | | | | | | | | | | |
| | Gearbox length (L) mm | | 32.7 | 43 | | 53.5 | | | 64 | | | |
| | Gearbox Level | | Level 1 | Level 2 | | Level 3 | | | Level 4 | | | |
| | Weight (g) | | 750 | 820 | | 900 | | | 980 | | | |
| 12V | No-load | Speed/rpm | 2700 | 730 | 525 | 200 | 140 | 100 | 53 | 38 | 27 | 19 |
| | | Current/A | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 | 2.6 |
| | Rated | Speed/rpm | 2400 | 650 | 467 | 176 | 126 | 89 | 47 | 33 | 24 | 14 |
| | | Power/W | 66 | 66 | 66 | 66 | 66 | 66 | 66 | 66 | 66 | 66 |
| | | Torque/kgf.cm | 1.3 | 4.8 | 7 | 19 | 27 | 38 | 69 | 101 | 135 | 155 |
| | | Current/A | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 | 5.5 |
| | Stall | Torque/kgf.cm | 3.7 | 4.8 | 7 | 21 | 27 | 90 | 110 | 221 | 330 | 460 |
| | | Current/A | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 |
| | | Power/W | 120 | 120 | 120 | 120 | 120 | 120 | 120 | 120 | 120 | 120 |
| 24V | No-load | Speed/rpm | 2700 | 730 | 525 | 200 | 140 | 100 | 53 | 38 | 27 | 19 |
| | | Current/A | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 | 1.3 |
| | Rated | Speed/rpm | 2400 | 650 | 467 | 176 | 126 | 89 | 47 | 33 | 24 | 14 |
| | | Power/W | 90 | 90 | 90 | 90 | 90 | 90 | 90 | 90 | 90 | 90 |
| | | Torque/kgf.cm | 1.6 | 5.8 | 8 | 21 | 30 | 42 | 78 | 110 | 154 | 172 |
| | | Current/A | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 | 3.8 |
| | Stall | Torque/kgf.cm | 4.8 | 17 | 24 | 65 | 98 | 127 | 250 | 360 | 423 | 500 |
| | | Current/A | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 |
| | | Power/W | 240 | 240 | 240 | 240 | 240 | 240 | 240 | 240 | 240 | 240 |

Attention: There may be a deviation of plus or minus 5% in the marked speed of the motor, which is a normal phenomenon. The motor is prohibited from being stuck in rotation! Do not directly connect to 220V AC power, otherwise the motor will be burned out!

**(c)** Hoja del vendedor (columna 189).

**Figura 3.3:** Ruedas y motores de tracción.

| Dato | Valor |
|---|---|
| Rueda | inflable, 150 mm de diámetro, 45 mm de ancho, 650 g |
| Motor | 42GM-895H planetario, 12 V, reducción 189:1 |
| Velocidad sin carga / nominal | 53 rpm / 47 rpm |
| Par nominal / trabado | 69 kgf · cm / 110 kgf · cm |
| Corriente sin carga / nominal / trabado | 2,6 A / 5,5 A / 10 A |
| Encoder | Hall, 11 pulsos por vuelta del motor, canales A y B, alimentado a 5 |
| Cuentas por vuelta de la rueda | $11 \times 4 \times 189 = 8316$ (cuadratura ×4) **MEDIR** |
| Velocidad máxima teórica | $53 \cdot \pi \cdot 0{,}15/60 \approx 0{,}42$ m/s |
| Velocidad máxima configurada | 0,25 m/s y 1 rad/s (`CFG_DIF` en `base_config.h`) |
| Velocidad de crucero del recorrido | 0,10 m/s |

**Por qué se cuentan 4 flancos**

El encoder tiene dos canales, A y B, desfasados un cuarto de período. Contando los flancos de subida y de bajada de los dos canales se obtienen 4 cuentas por pulso ("cuadratura ×4"), y el orden en que cambian A y B dice el sentido de giro. El PCNT del ESP32 hace esto por hardware.

## 3.3 El brazo

### 3.3.1 Tipo y geometría

El brazo es un manipulador **RRR** (tres articulaciones rotativas) del tipo **uArm / Dobot Magician**:

- **J1 (base)** gira todo el brazo alrededor de un eje vertical.
- **J2 (hombro)** levanta la barra principal $L_2$.
- **J3 (codo)** mueve, a través de un **mecanismo de cuatro barras**, el antebrazo $L_3$. El motor de J3 está abajo, en la base del brazo, no en el codo: así el brazo es más liviano.
- Un **paralelogramo** mantiene la herramienta **siempre horizontal**, apuntando hacia afuera en la dirección de J1. Por eso basta con 3 motores para posicionar la pinza.

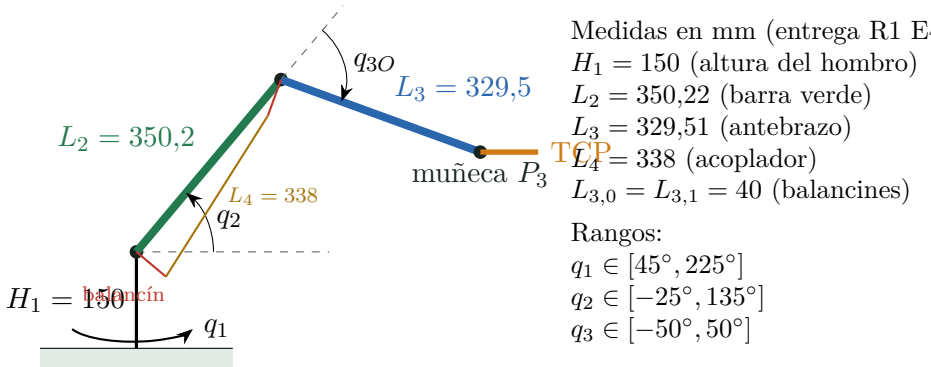



**Figura 3.4:** Esquema lateral del brazo (no a escala). El motor de J3 mueve el balancín rojo de abajo; el acoplador amarillo transmite el movimiento al codo. $q_{3O}$, el ángulo del antebrazo respecto de la prolongación de $L_2$, es una consecuencia de $q_2$ y $q_3$ por el cierre del cuatro barras.

**Balancín del cuatro barras**

`cinematica.py` usa balancines de 40 mm (E4). Según `ARQUITECTURA.md` §11, el balancín rojo se cambió a **60 mm** el 03/10 para que el mecanismo sea de Grashof. Verificar la medida real y, si cambió, actualizar `L30` en `ros2_ws/src/brazo_ik/brazo_ik/cinematica.py` y volver a correr sus pruebas.

### 3.3.2 Dónde va montado y qué alcanza

El brazo va **atrás a la izquierda, sobre el eje de tracción**, a unos 6 cm del costado ($y = 0{,}11$ m, preliminar). $q_1 = 0$ apunta hacia adelante y crece en sentido antihorario visto desde arriba. El rango de J1 (45° a 225°) va de adelante-izquierda, pasando por la izquierda y por atrás, hasta atrás-derecha: **nunca cruza el mástil de las cámaras**, que está adelante a la derecha. $q_1 = 90°$ (brazo hacia el costado izquierdo) es la marca del banco de pruebas y la pose de cero de J1.

La consigna de R1 define la zona de trabajo como una esfera de 30 cm de diámetro centrada a unos 35 cm del suelo. Con el robot centrado en el pasillo de la simulación, los limones quedan a unos 0,45 m del eje de J1.

### 3.3.3 Motores y transmisiones

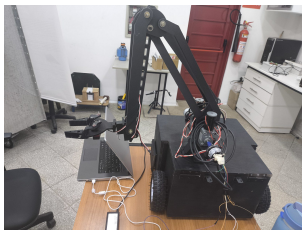
| | **J1 (base)** | **J2 (hombro)** | **J3 (c** |
|---|---|---|---|
| Motor | 36GP-555, 160 rpm | 36-555, 58 rpm | 36-555, |
| Reductora del motor | 50:1 | 139:1 | 139 |
| Transmisión | correa GT2 18 → 90 (5:1) | helicoidales 15 → 75 (5:1) | helicoidales 15 |
| Velocidad máx. de la articulación | 32 rpm (192 °/s) | 11,6 rpm (70 °/s) | 11,6 rpm |
| Encoder | SCX-555, 16 pulsos por vuelta, cuadratura ×4, alimentado a **3,** | | |
| Cuentas por vuelta de la articulación | $3200 \times 5 = 16\,000$ | $8896 \times 5 = 44\,480$ | 44 4 |
| Resolución | 44,4 cuentas/° | 123,6 cuentas/° | 123,6 cu |
| Corriente (vacío / nominal / trabado) | | 0,5 A / 1,3 A / 10 A a 12 V | |
| Se autobloquea | no | **no** | **no** |
| Estado | medido (02/10) | llega 19–29/10 | llega 19- |

**J2 y J3 no se sostienen solos**

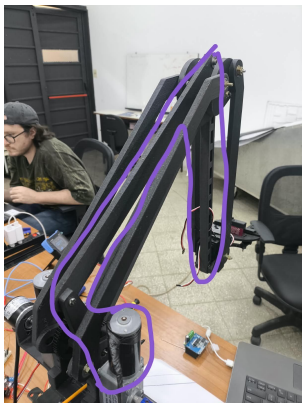
Los motores planetarios de J2 y J3 **no se autobloquean**: sin corriente, el brazo cae por su peso. Pasa al apretar la seta, al desarmar en rueda libre o al cortar los 12 V. Dejar siempre el brazo en reposo **apoyado en un tope** y no poner las manos debajo. Falta resolver cómo sostenerlo (contrapeso o resorte con lo que haya).



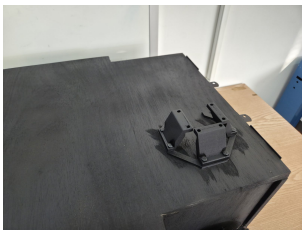

**(a)** 36GP-555 con encoder.



**(b)** J1 en su cero.



**(c)** Cuatro barras del codo.



**(d)** Base impresa del brazo.

**Figura 3.5:** Motores y partes del brazo.

Colores de los cables de los 36GP-555: **blanco** = motor +, **amarillo** = motor −, **verde** = encoder + (a 3,3 V), **azul** = GND, **negro** = canal A, **rojo** = canal B. El encoder tiene salida de colector abierto con pull-up de 5,1 kΩ en su plaquita: alimentado a 3,3 V, sus salidas quedan en 0 o 3,3 V, que el ESP32 tolera.

## 3.4 La pinza

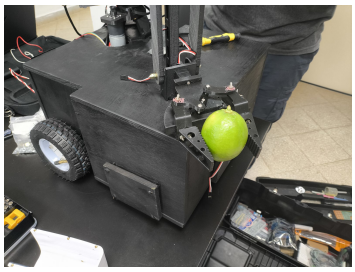
La pinza tiene dos servos **DSSERVO RDS3218** de 20 kg · cm, alimentados con 6 V por el buck 3:

- **Apertura** (GPIO 4 de la WROOM): 0 % = abierta, 100 % = cerrada (`/pinza/cierre`).
- **Giro J4** (GPIO 14): rota la pinza alrededor del eje de la herramienta, de −90° a +90° (`/pinza/giro`). Sirve para desprender el limón retorciendo el pedúnculo.

La cosecha es: **agarrar** (cerrar), **traccionar** (retroceder 1–2 cm), **girar** J4 y **retirar**. Hoy la misión hace solo agarrar y retirar; el giro queda para después.




**(a)** Servo RDS3218.



**(b)** Pinza con un limón.

**Figura 3.6:** La pinza.

**Pendiente de medir o probar**

Pulsos reales de cada servo (`PINZA_US_*`, `GIRO_US_*`), el porcentaje de cierre que agarra sin aplastar (`cierre` en la misión) y `largo_herramienta`: la distancia horizontal de la muñeca al centro del agarre (capítulo 10).

CAPÍTULO 4

# Alimentación y potencia

## 4.1 Dos fuentes separadas

El robot tiene **dos fuentes de energía que nunca se tocan** (salvo el GND común):

**LiPo 3S para los motores** 11,1 V nominal (9,9–12,6 V), 8000 mAh (88,8 Wh). Alimenta los cinco IBT-2 y el buck de los servos. Los motores son de 12 V y el IBT-2 acepta de 5,5 a 27 V, así que la LiPo va **directa, sin convertidor**.

**Power bank para la lógica** Xiaomi PB2020MI, 5 V 3 A por su cable USB-C. Alimenta la Raspberry Pi, y la Pi alimenta los dos ESP32 por USB.

**Por qué separadas**

Un motor trabado puede pedir 10 A y hacer caer la tensión de la batería. Si la Pi estuviera en la misma batería, se reiniciaría en medio de una cosecha. Con fuentes separadas, aunque los motores se queden sin energía (seta, fusible, batería baja), la Pi y los ESP32 siguen encendidos y saben lo que pasó.

## 4.2 El circuito, paso a paso

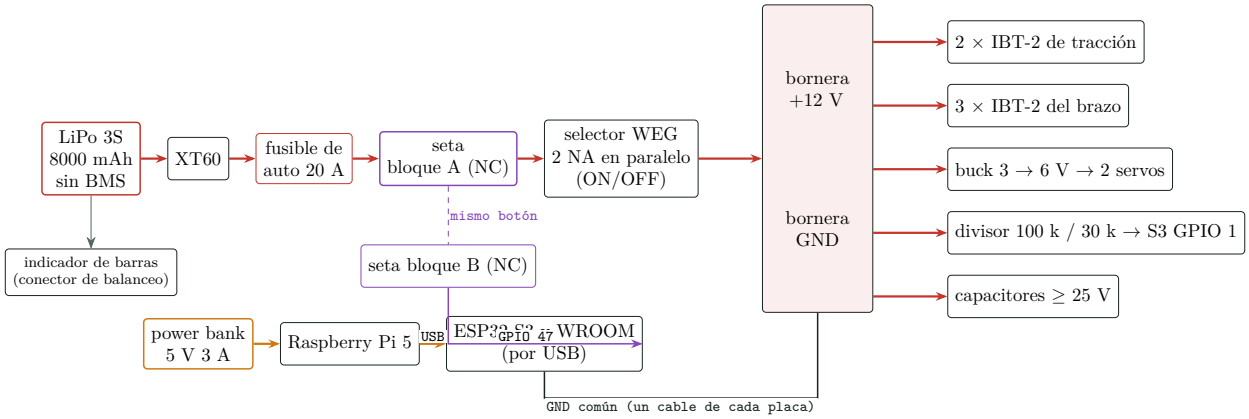



**Figura 4.1:** Circuito de potencia (v0.7, 10/10). Rojo: 12 V de motores. Naranja: 5 V de lógica. Violeta: seguridad.

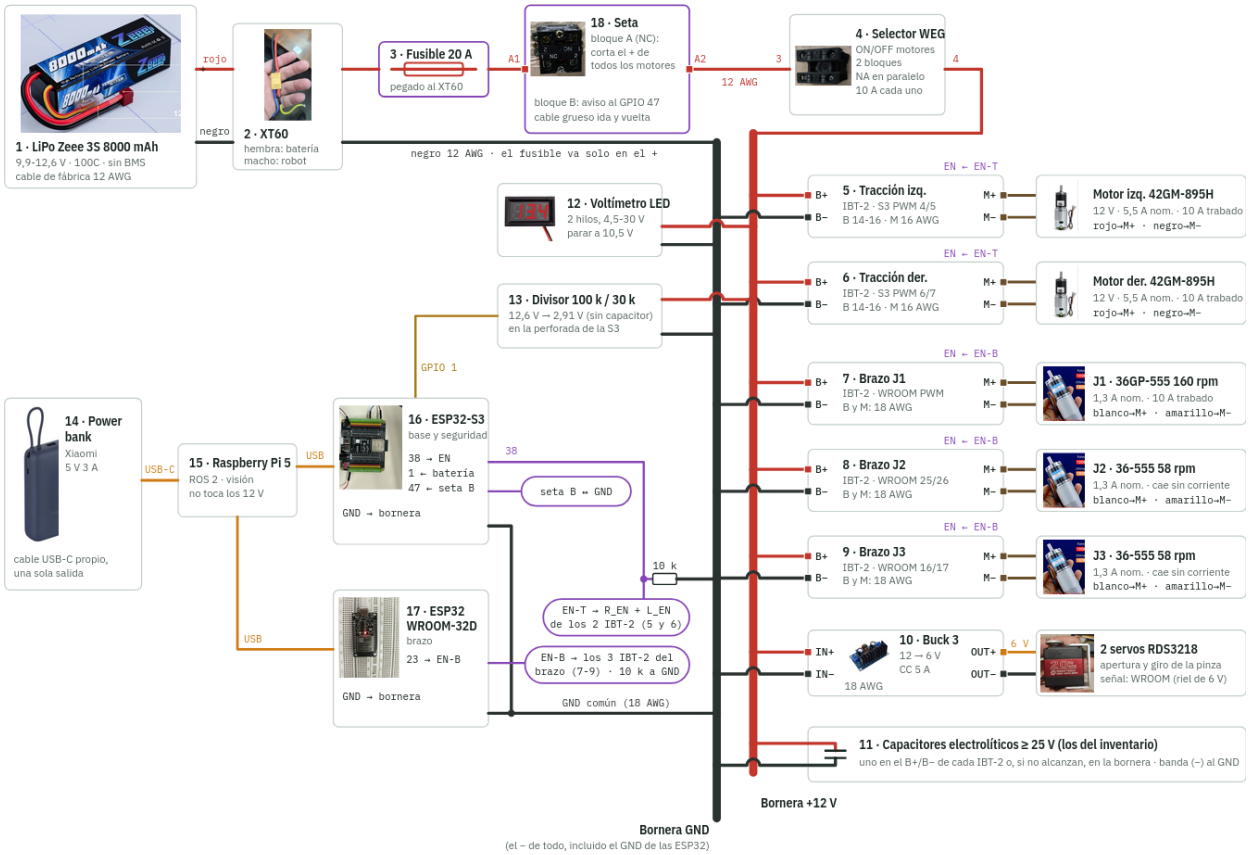
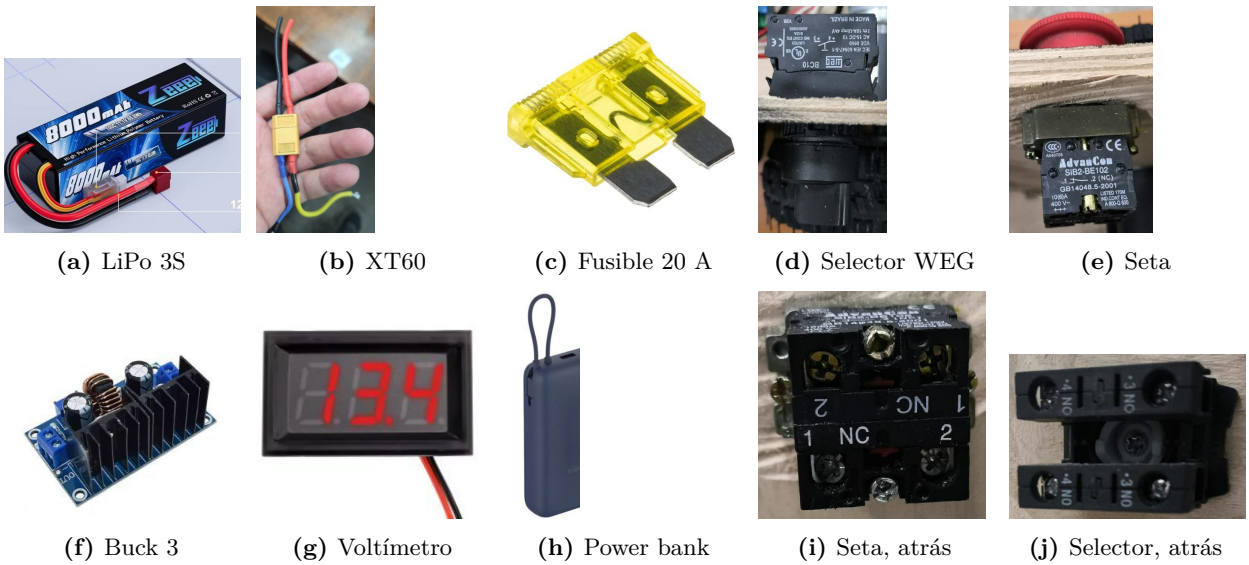



**Figura 4.2:** Cableado de potencia completo, con fotos y calibres (de `docs/html/potencia.html`). Los números coinciden con las fichas de la sección 4.3.

## 4.3 Cada pieza



**(a)** LiPo 3S **(b)** XT60 **(c)** Fusible 20 A **(d)** Selector WEG **(e)** Seta

**(f)** Buck 3 **(g)** Voltímetro **(h)** Power bank **(i)** Seta, atrás **(j)** Selector, atrás

**Figura 4.3:** Las piezas del circuito de potencia.

| Pieza | Qué hace y cómo se usa |
|---|---|
| LiPo Zeee 3S | 11,1 V, 8000 mAh, 100C, 493 g. **No tiene BMS**: nada la protege de la sobredescarga. Se carga solo con el cargador balanceador y fuera del robot. Guardada varios días, a ∼11,4 V (3,8 V por celda). Su conector Deans de fábrica se cambia por un **XT60** (no hay Deans hembra). |
| XT60 | Conector de la batería. Se enchufa **con el selector en OFF** y respetando la polaridad: los IBT-2 al revés se rompen. Al cambiar el Deans, cortar **de a un cable** (los dos juntos con la pinza hacen un corto). |
| Fusible de auto 20 A | Lámina ATO/ATC, apto para continua hasta 32 V. Va **pegado al XT60**: el cable antes del fusible no está protegido. **Protege contra cortocircuitos**; no salta con dos motores trabados ($2 \times 10$ A es justo su nominal). Cable del bus de al menos 14 AWG. |
| Selector WEG | ON/OFF diario de los motores. Dos bloques BC10 normalmente abiertos en paralelo. **Moverlo solo con los motores quietos**: dos contactos en paralelo no suman 20 A al abrir (el último en separarse corta todo). |
| Seta (bloque A) | Contacto normalmente cerrado en serie con el + del bus de 12 V, entre el fusible y el selector. Al apretarla, los cinco IBT-2 y el buck quedan sin tensión. Los motores quedan en rueda libre: **no frena**. Está pensada para alterna (`10(6)A 400V~`): solo para emergencias. |
| Seta (bloque B) | Otro contacto NC del mismo botón, entre el GPIO 47 de la S3 y GND. Avisa al firmware. **Nunca 12 V en este bloque**: queman el GPIO. |
| IBT-2 (×5) | Puentes H BTS7960: 2 de tracción y 3 del brazo. Entrada de potencia B+/B−, salida al motor M+/M−. |
| Buck 3 | XL4016 (4–38 V de entrada, 5 A). Baja 12 V a 6 V para los servos, con límite de corriente en 5 A. **Ajustarlo sin carga y con el multímetro** antes de conectar los servos. |
| Capacitores | Electrolíticos de $\geq 25$ V cerca de los IBT-2, para absorber picos. |
| Divisor 100 k / 30 k | Baja 12,6 V a 2,91 V para el ADC del GPIO 1 de la S3. Si alguna vez vuelve la batería 10S (36 V), **cambiarlo antes a 120 k / 8,2 k**: con 42 V el de 30 k pondría ∼9 V en el GPIO y lo quema. |
| Voltímetro / indicador | Muestra la tensión a la vista. Parar a 10,5 V. |
| Power bank | 20 000 mAh, 5 V 3 A. Una sola salida, cable incorporado (corto), arriba al lado de la Pi. No cargarlo mientras alimenta la Pi (puede cortar un instante). |

## 4.4 Consumo y autonomía

| Carga | Potencia |
|---|---|
| Tracción avanzando a 0,25 m/s ($2 \times 60\,\% \times 12$ V $\times$ 3 A) | ∼43 W |
| Tracción girando o en terreno malo | ∼70 W |
| Brazo moviéndose (J1–J3 + servos) | ∼26 W |
| Brazo quieto sosteniendo (J2/J3 contra el peso + pinza) | ∼6 W |
| Lógica: Pi 5 con YOLO (∼9 W), ESP32 (0,8 W), sensores (0,6 W) | ∼11 W |

Ciclo de cosecha de un limón (60 s): 20 s avanzando, 5 s maniobrando, 25 s con el brazo en movimiento y 10 s con el brazo quieto:

$$P_{\text{motores}} = \frac{20\,(43+6) + 5\,(70+6) + 25 \cdot 26 + 10 \cdot 6}{60} \approx 34{,}5 \text{ W}$$

Con el 80 % de la energía nominal como útil (71 Wh):

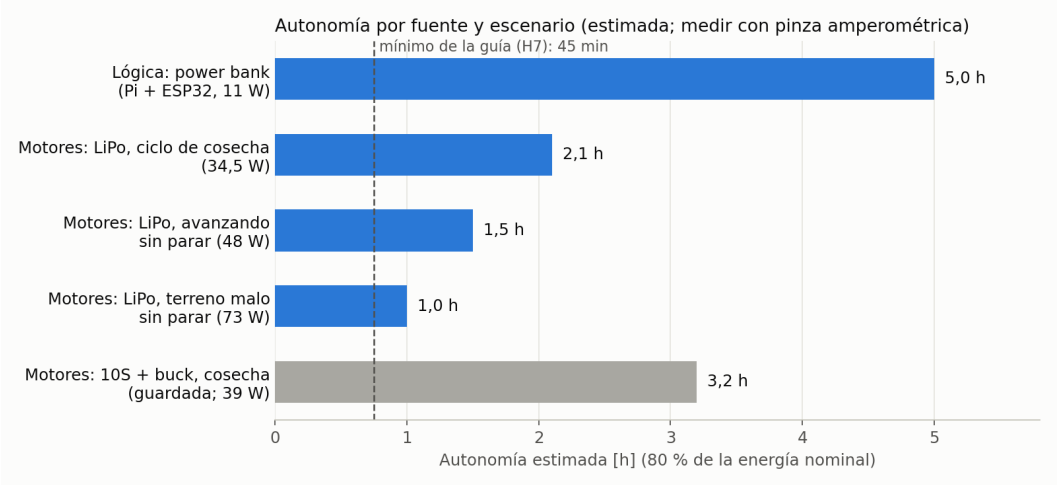



**Figura 4.4:** Autonomía estimada por fuente y escenario (de la entrega R2 E4). La guía pide al menos 45 min. El robot dura lo que dura la LiPo: ∼2,1 h de cosecha.

## 4.5 La batería sin BMS

**La LiPo no se protege sola**

- El firmware de la base mide la tensión y avisa `BATERIA_BAJA` a 10,5 V en `/base/estado`, pero **no frena**: hay que parar a mano.
- 10,5 V en total no garantizan 3,5 V en cada celda. Ni el ADC ni el indicador ven una celda desbalanceada. **Medir cada celda** con el balanceador o el tester antes y después de cada prueba.
- Calibrar la lectura del ADC contra el tester.
- Con la seta apretada, el divisor marca 0 V (`BATERIA_BAJA`): es normal.

## 4.6 Reglas para las pruebas

La revisión de Codex del 14/10 confirmó que el conexionado está bien, pero que varios componentes no tienen garantizado cortar con carga en continua. Por eso las pruebas son **acotadas y vigiladas**:

1. Mover el selector **solo con los motores quietos**.
2. Si se aprieta la seta, **mandar `desarmar` al brazo y confirmar que quedó desarmado antes de soltarla** (aunque `mision` esté corriendo).
3. **Nada de pruebas de motores trabados con la LiPo**: el fusible no salta con eso. Usar una fuente limitada en corriente.
4. Cables a las borneras con **cobre pelado y retorcido, sin estañar**: el estaño cede bajo el tornillo.
5. Siempre alguien con la mano en la seta o en el selector.
6. Apagar los 12 V antes de flashear o resetear un ESP32 (al reiniciar, los pines quedan un instante sin control).

CAPÍTULO 5

# Placas, pines y cableado

## 5.1 Vista general del cableado

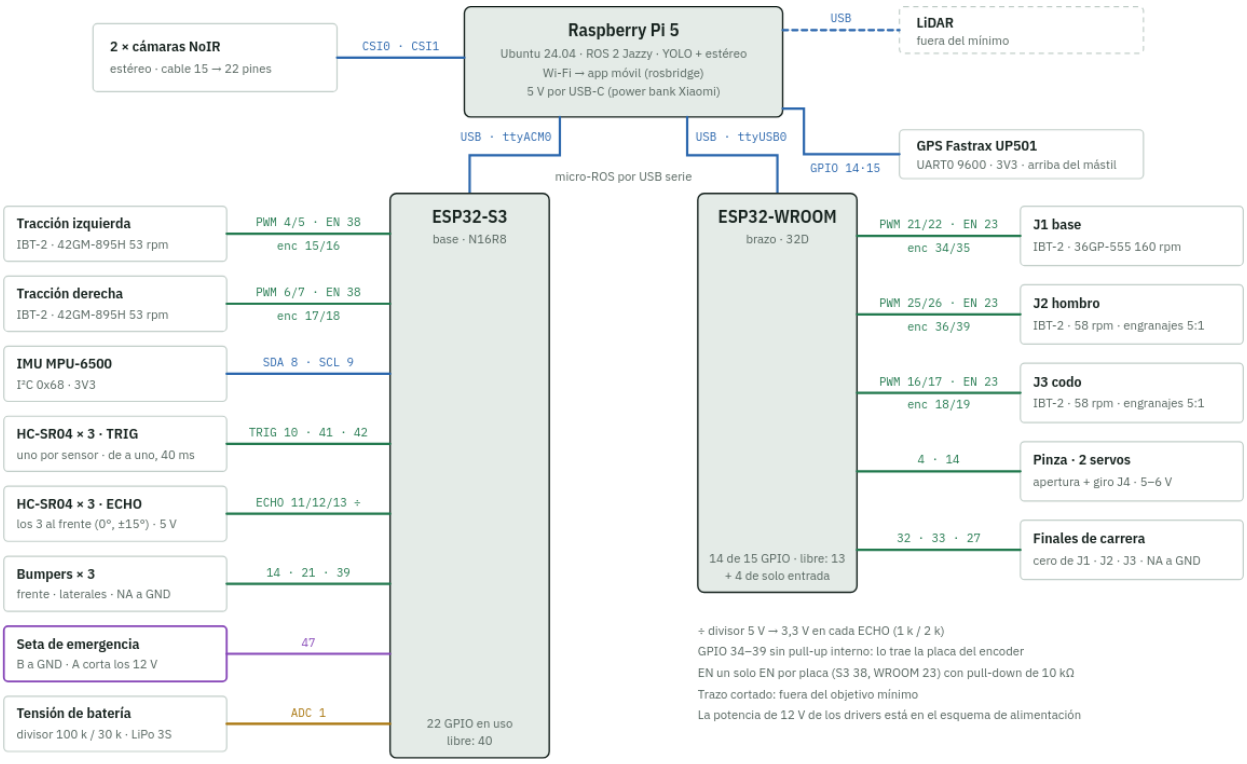



**Figura 5.1:** Qué va conectado a qué, con el número de GPIO en cada cable (de `docs/html/conexiones.html`). Azul: datos (USB, CSI, I²C, UART). Verde: GPIO y PWM. Ocre: analógico. Violeta: seguridad.

**Dónde está la verdad**

Los pines **solo** se definen en `firmware/base/include/pins.h` y `firmware/brazo/include/pins.h`. `docs/CONEXIONES.md` y las páginas de `docs/html/` los copian. Si hay una diferencia, manda `pins.h`. Las páginas HTML son interactivas: `docs/html/todo.html` junta todas, con cajas clickeables.

## 5.2 Tres ideas eléctricas que aparecen en todas partes

### 5.2.1 Los GPIO del ESP32 son de 3,3 V

Un pin del ESP32 se rompe si recibe más de 3,6 V. Hay tres cosas en el robot que sacan 5 V: el ECHO de los HC-SR04, los encoders de tracción y (si vuelve la 10S) la batería. Todas pasan por

un **divisor resistivo**:

$$V_{\text{GPIO}} = V_{\text{entrada}} \frac{R_{\text{abajo}}}{R_{\text{arriba}} + R_{\text{abajo}}}$$

- ECHO del HC-SR04: 1 kΩ arriba y 2 kΩ abajo: $5 \cdot \frac{2}{3} = 3{,}33$ V.
- Encoders de tracción: tienen un pull-up interno de 10 kΩ a 5 V; se agrega 20 kΩ a GND: $5 \cdot \frac{20}{30} = 3{,}33$ V.
- Batería: 100 kΩ arriba y 30 kΩ abajo: $12{,}6 \cdot \frac{27}{127} = 2{,}68$ V (el ADC lee hasta ∼3,1 V).

### 5.2.2 Pull-up y pull-down

Un pin de entrada "al aire" lee cualquier cosa. Un **pull-up** (resistencia a 3,3 V) lo deja en 1 cuando nada lo conecta; el interruptor lo lleva a 0 conectándolo a GND. Así funcionan los bumpers, la seta y los finales de carrera: **apretado = LOW**. El ESP32 tiene pull-ups internos, **salvo los GPIO 34 a 39 de la WROOM**, que son solo de entrada y no tienen: ahí van los encoders, que traen su propio pull-up de 5,1 kΩ.

Un **pull-down** hace lo contrario. Los EN de los IBT-2 tienen uno de 10 kΩ: mientras el ESP32 arranca o se reinicia, sus pines flotan; el pull-down mantiene los puentes deshabilitados y los motores quietos.

### 5.2.3 La seta es normalmente cerrada (falla segura)

El bloque B de la seta une el GPIO 47 con GND cuando está suelta. Al apretarla, o si **se corta un cable**, el pin sube a HIGH por el pull-up y la S3 entra en EMERGENCIA. Un cable cortado se comporta como una seta apretada: es más seguro que al revés.

## 5.3 ESP32-S3: la base

Placa Freenove ESP32-S3-WROOM N16R8, con USB nativo (aparece en la Pi como `/dev/ttyACM0`). Va encima de una **placa de expansión Freenove** con una bornera por pin, y los componentes (resistencias) van en dos perforadas chicas, una sobre cada fila de borneras. Desde el 13/10 **no lleva capacitores**: bumpers y seta van directo.

| GPIO | Conecta a | Nota |
|---|---|---|
| 4 · 5 | IBT-2 izquierdo RPWM · LPWM | PWM a 20 kHz, 10 bits |
| 6 · 7 | IBT-2 derecho RPWM · LPWM | 20 kHz |
| 38 | R_EN + L_EN de los dos IBT-2 | pull-down 10 kΩ |
| 15 · 16 | Encoder izquierdo A · B | amarillo · blanco; 20 kΩ a GND |
| 17 · 18 | Encoder derecho A · B | amarillo · blanco; 20 kΩ a GND |
| 8 · 9 | IMU SDA · SCL | I²C 400 kHz, dirección 0x68 |
| 10 · 41 · 42 | TRIG del HC-SR04 frente · izquierda · derecha | disparados de a uno, cada 40 ms |
| 11 · 12 · 13 | ECHO frente · izquierda · derecha | divisor 1 k / 2 k |
| 14 · 21 · 39 | Bumpers frente · izquierda · derecha | NA a GND, pull-up interno |
| 47 | Seta, bloque B (NC) | a GND; HIGH = apretada o cable cortado |
| 1 | Divisor de batería 100 k / 30 k | ADC1 canal 0 |
| 40 | libre | reservado para un 4.º bumper |

Prohibidos en la S3: 19/20 (USB), 26–37 (memoria flash y PSRAM). Evitar: 0/3/45/46 (*strapping*: definen el modo de arranque), 43/44 (UART0), 2/48 (LED).

**LED RGB de la S3**

En la DevKitC-1 v1.1 el LED RGB está en el GPIO 38, que es el EN de la tracción. `pins.h` supone el 48. Verificar la placa real y no usar nunca el LED RGB en el firmware.

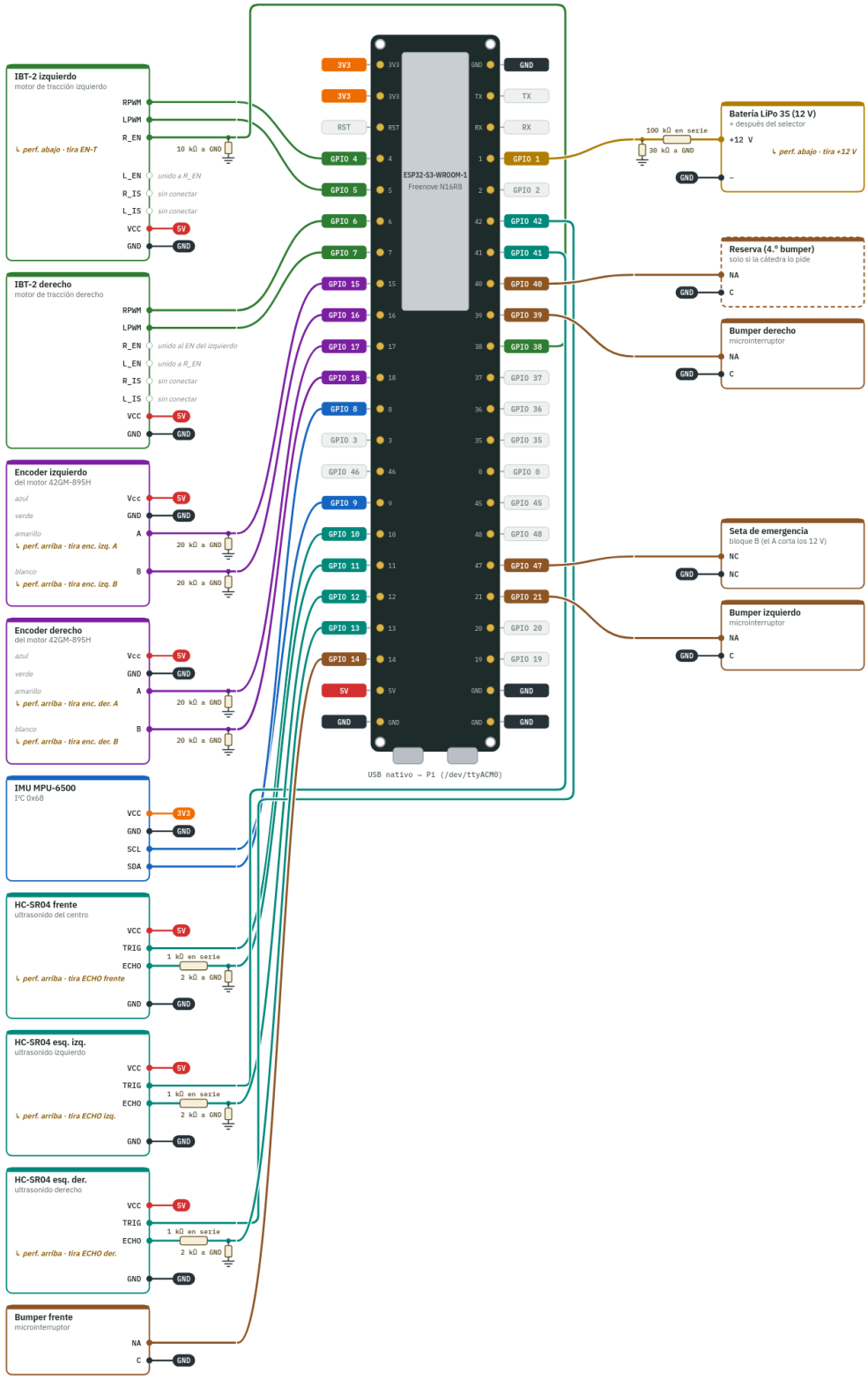
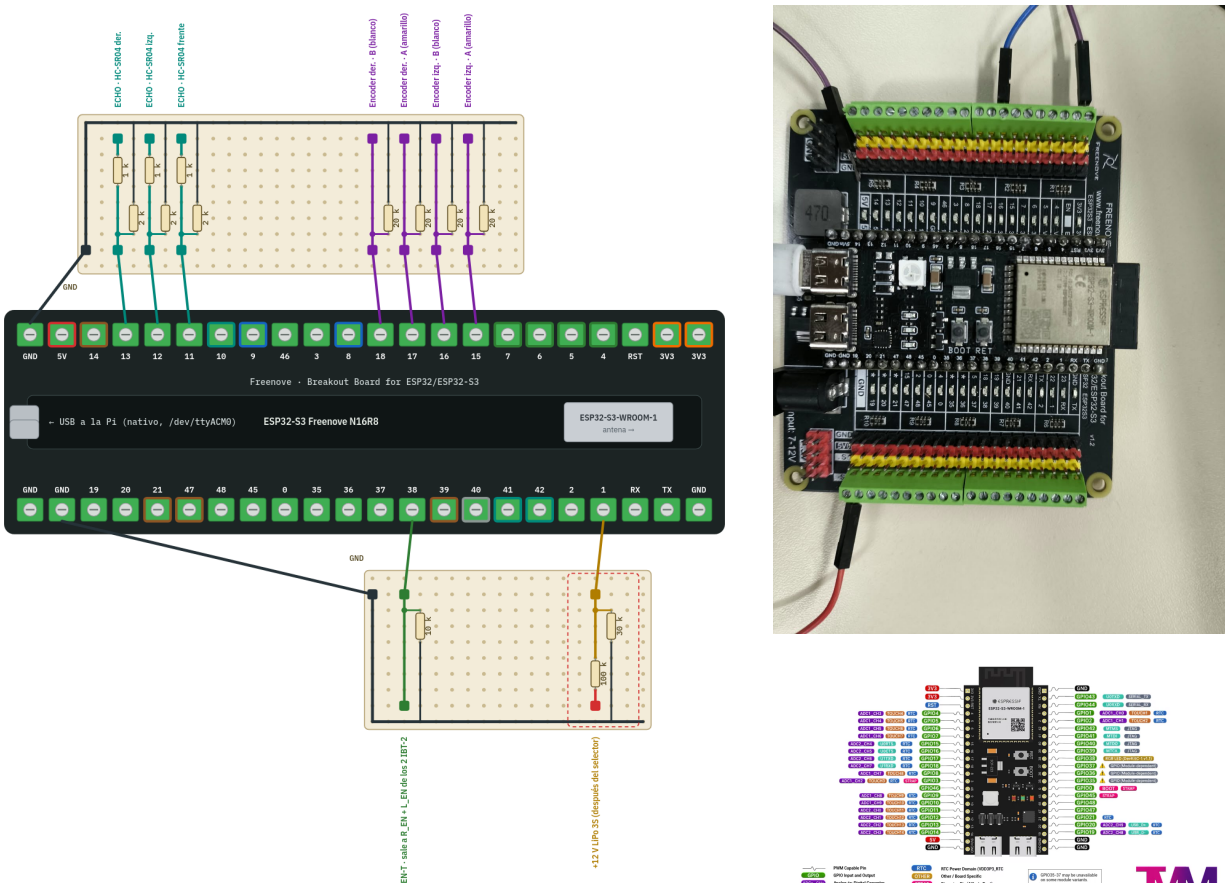



**Figura 5.2:** Cableado de la S3: cada periférico a un costado, con un cable de color de cada pin al pin de la ESP32 (de `docs/html/placa_s3.html`). Las resistencias se dibujan sobre el cable.




**(a)** La Freenove con las dos perforadas.

**(b)** Placa real y pinout.

**Figura 5.3:** Montaje de la S3.

## 5.4 ESP32-WROOM-32D: el brazo

Placa HW-394 de 30 pines (USB-C, conversor CH340: aparece como `/dev/ttyUSB0`), enchufada en un **shield de borneras**, más una perforada chica de distribución con rieles de 5 V, 3,3 V, EN, GND y 6 V. Pines v0.5 (12/10).

| GPIO | Conecta a | Nota |
|---|---|---|
| 21 · 22 | J1 IBT-2 RPWM · LPWM | 20 kHz (probado) |
| 25 · 26 | J2 IBT-2 RPWM · LPWM | |
| 16 · 17 | J3 IBT-2 RPWM · LPWM | |
| 23 | R_EN + L_EN de los 3 IBT-2 | pull-down 10 kΩ (la única resistencia) |
| 34 · 35 | Encoder J1 A · B | solo entrada; pull-up de la placa del encoder |
| 36 (VP) · 39 (VN) | Encoder J2 A · B | solo entrada; pull-up de la placa del encoder |
| 18 · 19 | Encoder J3 A · B | |
| 4 | Servo de apertura | 50 Hz |
| 14 | Servo de giro (J4) | saca pulsos al arrancar: el servo puede moverse un instante |
| 32 · 33 · 27 | Finales de carrera J1 · J2 · J3 | NA a GND, pull-up interno |
| 13 | libre | reserva |

Evitar en la WROOM: 6–11 (flash), 0/2/5/12/15 (*strapping*), 1/3 (UART0 = USB).

**Un solo EN para las tres articulaciones**

Los tres IBT-2 del brazo comparten el EN (GPIO 23). Consecuencia: no se puede dejar una articulación en rueda libre mientras otra se mueve. La que no se mueve queda **frenada** (puente habilitado con PWM 0). Rueda libre solo cuando las tres están libres.

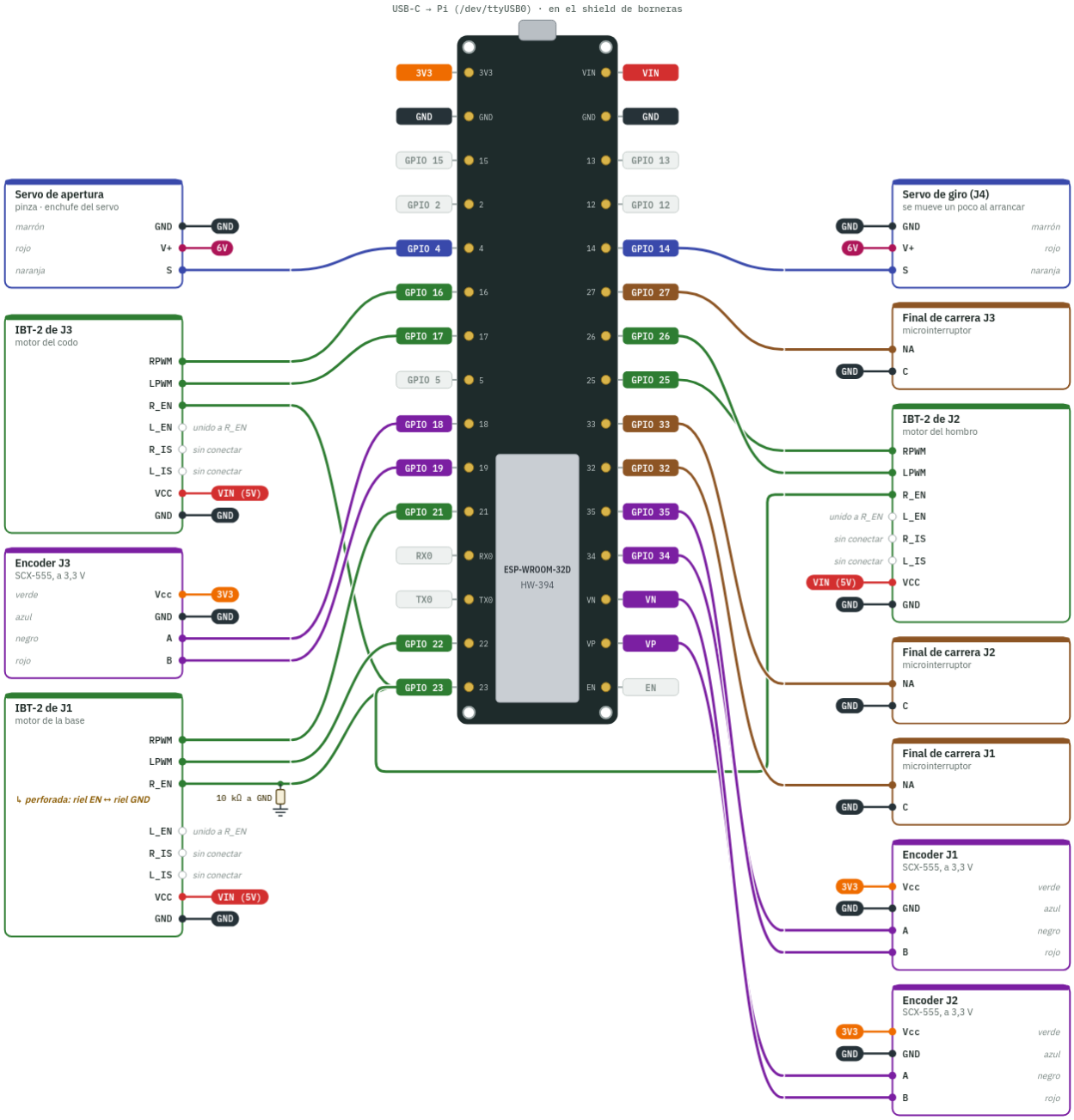
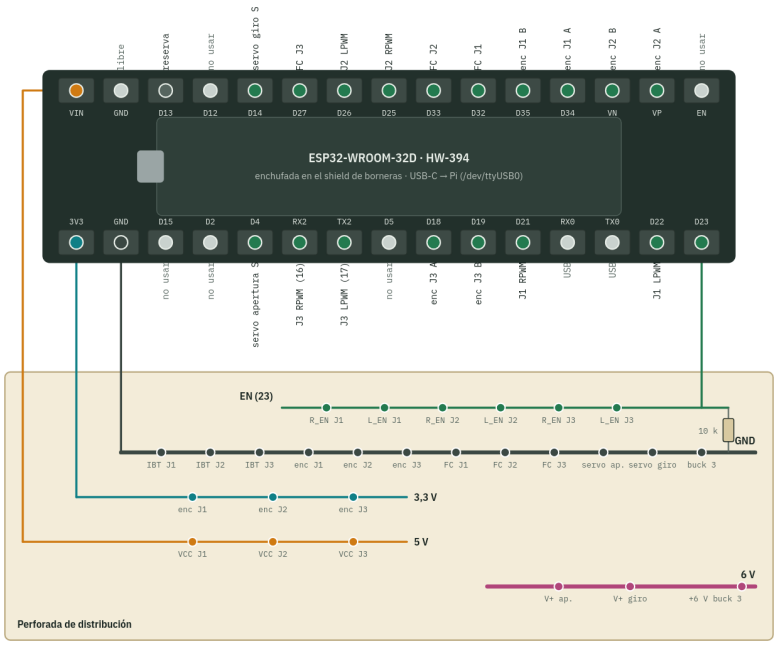
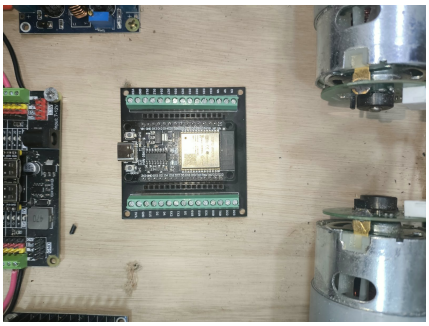



**Figura 5.4:** Cableado de la WROOM (de `docs/html/placa_wroom.html`).




**(a)** Shield y perforada de distribución.



**(b)** Shield de borneras real.

**Figura 5.5:** Montaje de la WROOM.

## 5.5 Raspberry Pi 5

| Puerto | Conecta a | Nota |
|---|---|---|
| CSI0 · CSI1 | Cámaras NoIR izquierda · derecha | cable plano de 15 a 22 pines |
| USB | ESP32-S3 (base) | micro-ROS (`/dev/serial/by-id/...`) |
| USB | ESP32-WROOM (brazo) | micro-ROS a 115 200 baudios |
| GPIO 14 · 15 (pines 8 · 10) | GPS UP501: RX · TX del GPS | UART0, `/dev/ttyAMA0`, 9600 baudios, 3,3 V |
| USB-C | Power bank | 5 V 3 A |

La Pi va **arriba del mástil, junto a las cámaras**: el cable CSI es corto y sensible a interferencias, mientras que el USB aguanta 1–2 m. Bajan por el mástil los dos USB hacia los ESP32. Los GPIO se atornillan en un HAT pasivo de borneras.

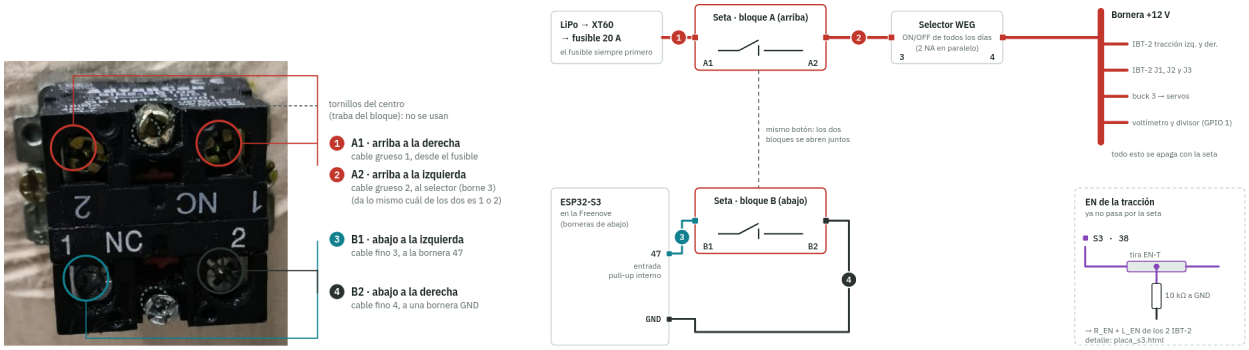
**Por qué usar `/dev/serial/by-id/`**

Los nombres `/dev/ttyACM0` y `/dev/ttyUSB0` pueden cambiar (la S3 pasa a veces de ACM0 a ACM1 al reiniciarse). Los enlaces de `/dev/serial/by-id/` tienen el nombre del fabricante y no cambian: usarlos en los agentes.

**Corriente de los USB**

Sin la fuente oficial de 5 A, la Pi limita a 600 mA el total de los USB **y el ventilador**. Alcanza para los dos ESP32 (unos 400 mA con sensores). No activar `usb_max_current_enable=1`. Verificar con la visión corriendo 10 min: `vcgencmd get_throttled` tiene que dar `0x0`.

## 5.6 La seta, cable por cable




**(a)** Qué tornillo es cuál. **(b)** Los 4 cables.

**Figura 5.6:** La seta (de `docs/html/seta.html`). Bloque A (arriba): cables 1 y 2, en el + del bus de 12 V. Bloque B (abajo): cables 3 y 4, entre el GPIO 47 y GND.

Antes de cablearla, **probar con el multímetro qué contactos son NC**. Prueba: con la seta apretada, 0 V en la bornera +12 V y `armar` tiene que fallar; al soltarla, las ruedas no arrancan solas.

CAPÍTULO 6

# Firmware de los ESP32

## 6.1 Organización del código

El firmware está en `firmware/` y se compila con **PlatformIO** (`~/.platformio/penv/bin/pio`). Hay un proyecto por placa y una carpeta de bibliotecas compartidas:

| Carpeta | Qué tiene |
|---|---|
| `firmware/lib/ControlArticulacion` | Control de un motor (cascada posición → velocidad), **sin Arduino** |
| `firmware/lib/EnlaceMicroRos` | Sesión con el agente: ping, armado, reconexión |
| `firmware/lib/BTS7960` | Driver del IBT-2 (dos PWM + EN) |
| `firmware/lib/EncoderPCNT` | Encoder por el contador de pulsos del ESP32 |
| `firmware/lib/HCSR04` | Los 3 ultrasonidos, sin bloquear (ECHO por interrupción) |
| `firmware/lib/MPU9250` | IMU (sirve para la MPU-6500) |
| `firmware/base/` | Proyecto de la S3: `src/`, `include/pins.h`, `include/base_config.h` |
| `firmware/base/lib/BaseLogica` | Cinemática diferencial, odometría y seguridad de la base, **sin Arduino** |
| `firmware/brazo/` | Proyecto de la WROOM: `src/`, `include/pins.h`, `include/brazo_config.h` |

Reglas del código:

- Los pines están **solo** en `include/pins.h`; los parámetros, en `base_config.h` o `brazo_config.h`.
- Los drivers compartidos se enlazan con `symlink://../lib/X` (no se copian).
- La lógica que no depende de Arduino (control, cinemática, seguridad) se prueba en la PC con `pio test -e native`, con un motor simulado con el modelo de la entrega E5.

### 6.1.1 Entornos de compilación

Cada proyecto tiene tres entornos (`platformio.ini`):

**`native`** Corre las pruebas en la PC. Base: 20 pruebas. Brazo: 17.

**`esp32s3` / `esp32dev`** Firmware con **comandos por el monitor serie**. Para probar a mano.

**`ros`** El mismo firmware con **micro-ROS**: los comandos llegan por tópicos. Es el que se usa con la Pi.

```
PIO=~/.platformio/penv/bin/pio
cd firmware/base                    # o firmware/brazo
$PIO test -e native                 # pruebas de la lógica en la PC
$PIO run                            # compilar (modo serial)
$PIO run -t upload -t monitor       # flashear y abrir el monitor (¡12 V APAGADOS!)
$PIO run -e ros -t upload           # flashear la versión micro-ROS (detener antes el agente)
```

## 6.2 Cómo corre el programa

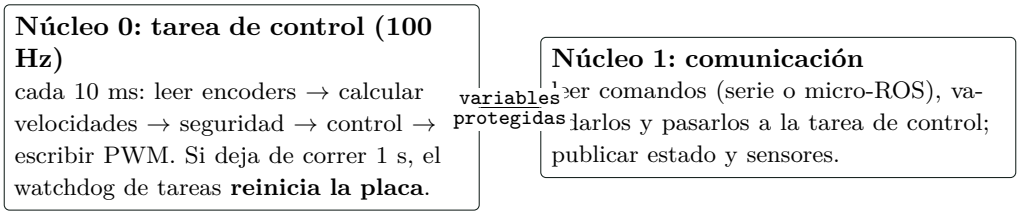



**Figura 6.1:** Las dos mitades del firmware. El control nunca espera a la comunicación.

## 6.3 El control de un motor: cascada posición → velocidad

La entrega E5 identificó cada motor como un **sistema de primer orden** de duty (%) a velocidad (rpm):

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{K}{\tau s + 1} \qquad \text{(J1: } K = 1{,}58 \text{ rpm/\%}, \ \tau = 0{,}065 \text{ s)}$$

Como el modelo es de **velocidad**, el control es en **cascada**: un lazo externo de posición genera la velocidad que tiene que seguir un lazo interno de velocidad.

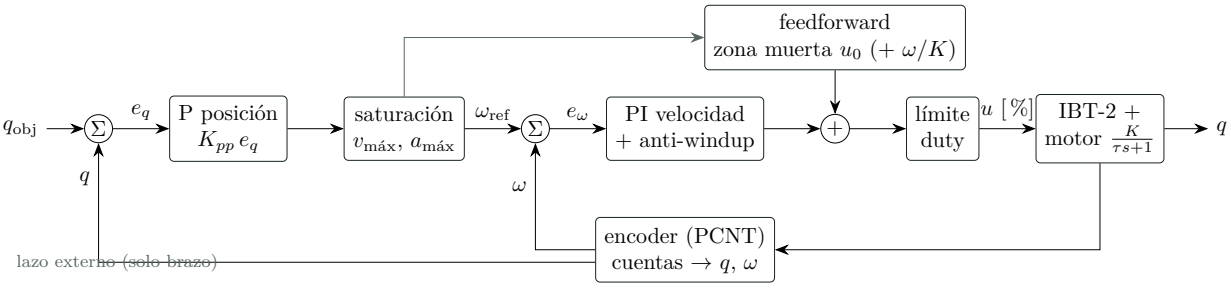



**Figura 6.2:** Control en cascada de `ControlArticulacion`. Las ruedas usan solo el lazo interno (velocidad).

Lo que hace la tarea de control en cada paso (10 ms), para una articulación del brazo:

1. **Error de posición**: $e_q = q_{\text{obj}} - q$.
2. **Llegada con histéresis**: si $|e_q| <$ tolerancia (0,3° en J1), se considera que llegó y el motor queda frenado; recién se vuelve a mover si el error supera el doble (0,6°). Así no "tiembla" alrededor del objetivo.
3. **Velocidad deseada**: $\omega_{\text{des}} = K_{pp}\, e_q$, saturada a $v_{\text{máx}}$ (30 °/s en J1) y con un mínimo $v_{\text{mín}}$ (3 °/s) para que no se clave a 1° del objetivo.
4. **Rampa**: $\omega_{\text{ref}}$ se acerca a $\omega_{\text{des}}$ cambiando como mucho $a_{\text{máx}}\, \Delta t$ (60 °/s²). El resultado es un perfil de velocidad aproximadamente **trapezoidal**.
5. **Lazo de velocidad**: $e_\omega = \omega_{\text{ref}} - \omega$, y

   $$u = \underbrace{\operatorname{sgn}\cdot u_0 + \tfrac{\omega_{\text{ref}}}{K}}_{\text{feedforward}} + K_p\, e_\omega + K_i \int e_\omega \, dt$$

   donde $u_0$ es la **zona muerta** (el duty mínimo para que el motor arranque: 17 % en J1). En J1 se usa solo la zona muerta como feedforward (`ffSoloZonaMuerta`).
6. **Anti-windup condicional**: si $u$ ya está saturado en el sentido del error, el integrador no acumula.

7. **Límite de duty**: 40 % en la puesta en marcha.

### 6.3.1 Parámetros de J1 (medidos el 02/10)

| Parámetro | Valor | Significado |
|---|---|---|
| Cuentas por vuelta del motor | 3200 | 16 PPR × 4 × 50 |
| Relación de la correa | 5 | 18 → 90 dientes |
| Rango | 45° a 225° | marco de `brazo_ik` |
| Modelo | $K = 1{,}58$, $\tau = 0{,}065$ s | E5 |
| Zona muerta | 17 % | medida con el brazo |
| PI de velocidad | $K_p = 0{,}659$ %/rpm, $K_i = 4{,}8$ | |
| $K_{pp}$, $v_{máx}$, $a_{máx}$ | 2 1/s, 30 °/s, 60 °/s² | |
| Tolerancia de llegada | 0,3° | |
| Atasco | duty ≥ 35 % y < 1,7 rpm durante 0,3 s | → falla |
| Signos | `ENC_INVERTIDO` y `MOTOR_INVERTIDO` en `true` | |

**Pendiente de medir o probar**

J2 y J3 arrancan con los parámetros de J1 llevados a 58 rpm ($K \times 58/160$; $K_p$ y $K_i \times 160/58$). Hay que medirlos con los motores nuevos: signos, zona muerta, $K$, $\tau$ y PI (pasos 0–3 de `pruebas-esp32/10`).

## 6.4 Firmware de la base (S3)

### 6.4.1 Velocidad de las ruedas

`/cmd_vel` trae $(v, \omega)$. La S3 aplica la rampa, calcula las rpm de cada rueda con la cinemática diferencial (figura 3.2) y cada rueda tiene su propio lazo PI de velocidad (el mismo `ControlArticulacion`, en modo velocidad). Si una rueda falla, las dos se frenan en el mismo paso.

### 6.4.2 Odometría

Con las cuentas de los encoders, cada 10 ms se calcula cuánto avanzó cada rueda ($\Delta s_i$, $\Delta s_d$) y se integra la pose $(x, y, \theta)$ en el marco `odom`:

$$\Delta s = \frac{\Delta s_d + \Delta s_i}{2}, \quad \Delta\theta = \frac{\Delta s_d - \Delta s_i}{b}, \quad x \mathrel{+}= \Delta s \cos\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad y \mathrel{+}= \Delta s \sin\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad \theta \mathrel{+}= \Delta\theta$$

Se publica en `/odom` a 25 Hz. Es buena para distancias cortas en línea recta; en giros cerrados las ruedas patinan y el ángulo se desvía. Por eso el recorrido autónomo toma el **rumbo del giróscopo** y solo el **avance** de las ruedas (capítulo 11).

### 6.4.3 Seguridad de la base

Se decide en la S3, sin esperar a la Pi:

| Causa | Qué hace | Cómo se sale |
|---|---|---|
| Seta apretada (o cable cortado) | EMERGENCIA: puentes deshabilitados (EN = 0) | Soltar la seta, `r` y `armar` |
| Inclinación (roll o pitch) > 25° | VUELCO: frena | Nivelar la base y `r` |
| IMU sin muestras por más de 200 ms | No se mueve (no puede vigilar el vuelco) | Solo, al volver los datos |
| Bumper del frente | No avanza; solo puede alejarse en línea recta | Solo, al soltarse |
| Bumper lateral | No gira (el costado barrería el objeto) | Solo, al soltarse |
| Ultrasonido del frente sin eco | No avanza; retroceder y girar sí | Solo, al volver el eco |
| Obstáculo al frente < 0,20 m | No avanza | Solo |
| Obstáculo entre 0,20 y 0,50 m | Tope de velocidad que baja linealmente a 0 | Solo |
| Sin `/cmd_vel` por 300 ms | Frena y anula el comando vigente | Con un comando nuevo |
| Batería < 10,5 V | Aviso `BATERIA_BAJA` (no frena) | — |
| Rueda atascada o encoder con saltos | Base detenida | `r` |

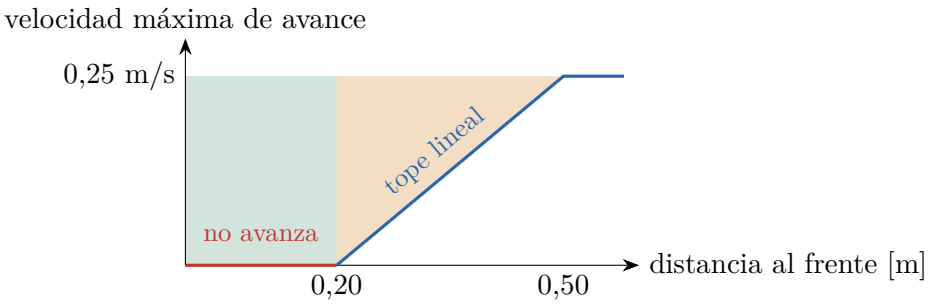



**Figura 6.3:** Tope de velocidad por cercanía de un obstáculo (HC-SR04 del frente). Retroceder y girar siempre se permite.

### 6.4.4 Comandos por el monitor serie (base)

| Comando | Qué hace |
|---|---|
| `e` | Estado: modo, seguridad, odometría y ruedas |
| `sens` | Sensores: IMU, distancias, bumpers, seta, batería |
| `v <v> <w> [s]` | Mover con rampa (m/s, rad/s) por `s` segundos (por defecto 1, máximo 3). Ej.: `v 0.1 0` |
| `w <izq> <der> [s]` | rpm de cada rueda (solo el lazo de velocidad). Ej.: `w 20 20` |
| `d <izq> <der>` | Lazo abierto en % (máximo 40); se corta a los 300 ms si no se repite |
| `s` | Detener (freno) |
| `l` | Liberar: puentes deshabilitados (rueda libre) |
| `r` | Rearmar tras seta o vuelco (con la causa despejada) y borrar fallas |
| `cal` | Calibrar el giróscopo (robot quieto, 2 s) |
| `odo0` | Odometría a cero |
| `log [hz]` | CSV para graficar (t, v, w, rpm de referencia y medidas, duty, x, y, $\theta$); `log 0` apaga |
| `wd <ms>` | Watchdog de comandos (0 = apagado; con la Pi, 300) |

| Comando | Qué hace |
|---|---|
| `?` | Ayuda |

## 6.5 Firmware del brazo (WROOM)

### 6.5.1 El cero y el homing

El encoder solo cuenta **cuánto se movió**, no dónde está. Al encender, el firmware no sabe el ángulo de cada articulación y **rechaza todo movimiento en posición** hasta tener un cero. Hay dos formas:

**Cero manual (`z`)** Se lleva el brazo a mano (o con `d`) a la pose de referencia (`POSE_CERO` = 90°, 90°, 0°) y se manda `z`: "el brazo **está** en esa pose".

**Homing (`home <n>`)** La articulación `n` va despacio (10 rpm del motor) hacia su final de carrera. Cuando lo toca (3 lecturas seguidas en bajo), fija su ángulo en `HOME_Q` (**MEDIR**). De a una articulación. Se cancela si se pasa del recorrido máximo (rango + 10°), del tiempo (25 s) o si aparece una falla. Rechaza empezar con el final de carrera ya apretado.

### 6.5.2 Protecciones del brazo

- Sin cero, no acepta movimientos en posición.
- Los objetivos se recortan a los límites de la E4. Fuera de los límites + 5° → falla y freno.
- Atasco (duty alto sin avanzar) → falla y freno. También detecta un encoder desconectado.
- Lecturas imposibles del encoder se descartan; 5 seguidas → falla.
- `s`, el fin de `d`/`v` y el watchdog **frenan hasta que el eje queda quieto** y recién ahí sostienen la posición (no vuelven hacia atrás por la inercia).
- `d` limitado a 30 % y se corta a los 300 ms si no se repite; `v` hasta 30 rpm y 5 s.
- Los servos no reciben pulsos hasta el primer `pinza`/`giro`.
- Watchdog de tareas: si la tarea de control se cuelga 1 s, la placa se reinicia.

### 6.5.3 Comandos por el monitor serie (brazo)

| Comando | Qué hace |
|---|---|
| `e` | Estado de las articulaciones |
| `z [n]` | Cero manual: el brazo está en la pose de referencia (todas o la `n`) |
| `home <n>` | Homing con final de carrera de la articulación `n` |
| `m <q1> <q2> <q3>` | Mover las tres (grados). Ej.: `m 90 90 0` |
| `j <n> <q>` | Mover una articulación. Ej.: `j 2 45` |
| `v <n> <rpm> [s]` | Solo el lazo de velocidad (rpm del motor, máx. 30) por `s` segundos |
| `d <n> <pct>` | Lazo abierto (máx. 30 %); se corta a los 300 ms si no se repite |
| `s` | Detener: frena y, ya quieto, sostiene la posición |
| `l` | Liberar: puentes deshabilitados (rueda libre: **J2 y J3 caen**) |
| `r` | Borrar fallas (queda detenido) |
| `pinza <0-100>` | 0 = abierta, 100 = cerrada |

| Comando | Qué hace |
|---|---|
| `giro <grados>` | Giro de la pinza (J4), −90 a 90 |
| `log <n> [hz]` | CSV de la articulación `n` (t, q objetivo, q, $\omega$ ref, $\omega$, duty) |
| `wd <ms>` | Watchdog de comandos |
| `?` | Ayuda |

Con micro-ROS, **los mismos comandos** se mandan como texto por `/base/comando` o `/brazo/comando`, con las mismas validaciones (capítulo 8).

CAPÍTULO 7

# ROS 2 desde cero

Este capítulo explica ROS 2 con los ejemplos del propio robot. Al terminarlo se puede mirar qué pasa dentro del robot, mandarle órdenes a mano y escribir un nodo propio.

## 7.1 Los conceptos

### 7.1.1 Nodos, tópicos y mensajes

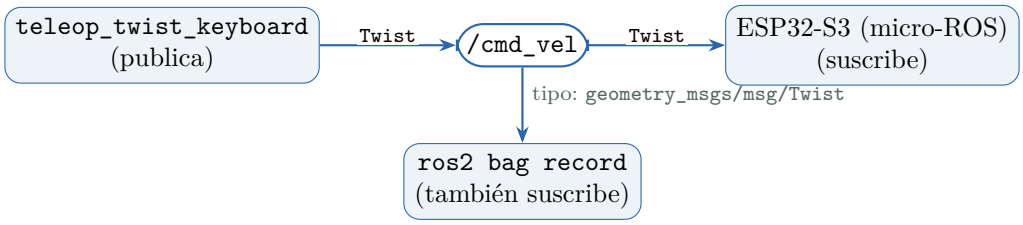



**Figura 7.1:** Un tópico es un canal con nombre. Quien publica no sabe quién escucha, y puede haber varios de cada lado.

**Nodo** Un programa que participa del sistema: `brazo_ik`, `mision`, el detector YOLO, cada ESP32.

**Tópico** Un canal con nombre, como `/cmd_vel`. Cualquier nodo puede publicar en él o suscribirse.

**Mensaje** Lo que viaja por el tópico. Tiene un **tipo fijo** con campos definidos. Por ejemplo `geometry_msgs/msg/Twist` tiene `linear` (x, y, z) y `angular` (x, y, z). En `/cmd_vel` solo se usan `linear.x` (avance, m/s) y `angular.z` (giro, rad/s).

**Publicar / suscribir** Mandar mensajes / recibirlos. Es asincrónico: el que publica no espera respuesta.

### 7.1.2 Otras piezas

**Parámetros** Valores de configuración de un nodo (`largo_herramienta`, `marco`...). Se pasan al arrancar (`-ros-args -p nombre:=valor`) o desde un *launch*.

**Launch** Un archivo Python que arranca varios nodos con sus parámetros de una vez (`mision.launch.py`, `sim.launch.py`, `limon.launch.py`).

**TF** Árbol de marcos de coordenadas. Cada mensaje con posición dice en qué marco está (`header.frame_id`), y TF sabe convertir de un marco a otro. Ej.: el limón llega en `camera` y el brazo lo necesita en `brazo_base`.

**Paquete** Carpeta con nodos, *launch* y pruebas, que se compila con `colcon`. En este repo: `brazo_ik`, `mision`, `simulacion` (en `ros2_ws/src/`); en el repo de visión, `yolo_ros`.

**Workspace** Carpeta con un `src/` de paquetes. `colcon build` crea `build/`, `install/` y `log/`.

**Descubrimiento (DDS)** Los nodos se encuentran solos por la red local. Todos los que estén en la misma LAN y el mismo `ROS_DOMAIN_ID` (0 por defecto) se ven entre sí.

**Cualquier computadora de la red puede mover el robot**

Una notebook con ROS 2 en la misma red Wi-Fi que la Pi ve los tópicos de los ESP32 reales y los puede comandar. La simulación se aísla con `export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST` en cada terminal (el *launch* de la simulación ya lo pone). Con el robot real, no dejar nodos corriendo con el brazo armado si no se usan.

## 7.2 Preparar una terminal

Cada terminal nueva necesita "cargar" ROS 2 y los workspaces que se vayan a usar:

```
source /opt/ros/jazzy/setup.bash                    # ROS 2 base (siempre)
source ~/microros_ws/install/local_setup.bash       # para correr el agente micro-ROS (en la Pi)
source ~/recolector_de_frutas/ros2_ws/install/setup.bash   # brazo_ik, mision, simulacion
source ~/yolo_ws/install/setup.bash                 # yolo_ros (visión, en la Pi)
```

Compilar los paquetes del robot (después de cambiar algo):

```
cd ~/recolector_de_frutas/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install            # --symlink-install: los .py se enlazan, no hace falta
    recompilar al editarlos
source install/setup.bash
colcon test && colcon test-result --verbose   # correr las pruebas
```

Conectarse a la Pi:

```
ssh r2d2@100.116.96.71          # por Tailscale (desde cualquier red); en la LAN del lab:
    192.168.2.219
```

## 7.3 Mirar qué está pasando

| Comando | Para qué |
|---|---|
| `ros2 node list` | Qué nodos están corriendo |
| `ros2 topic list -t` | Qué tópicos existen y de qué tipo |
| `ros2 topic echo /base/estado -field data` | Ver los mensajes que llegan (`-field` muestra un solo campo) |
| `ros2 topic echo /odom -once` | Ver un solo mensaje y salir |
| `ros2 topic hz /odom` | A qué frecuencia llega (debería ser 25 Hz) |
| `ros2 topic info -v /cmd_vel` | Quién publica y quién está suscripto |
| `ros2 interface show geometry_msgs/msg/Twist` | Qué campos tiene un tipo de mensaje |
| `ros2 param list /brazo_ik` | Parámetros de un nodo |
| `ros2 param get /brazo_ik largo_herramienta` | Leer un parámetro |
| `rqt_graph` | Dibujo del grafo de nodos y tópicos (pide entorno gráfico) |
| `ros2 run tf2_ros tf2_echo brazo_base camera` | Transformación entre dos marcos |
| `ros2 bag record /odom /cmd_vel /imu/data_raw` | Grabar tópicos a un archivo (para analizar o para la entrega) |

| Comando | Para qué |
|---|---|
| `ros2 bag play <carpeta>` | Reproducir una grabación |

## 7.4 Publicar a mano: `ros2 topic pub`

La forma general es:

```
ros2 topic pub [--once | -r <Hz>] <tópico> <tipo> "<mensaje en YAML>"
```

- `-once`: publica una vez y sale (para comandos).
- `-r 10`: publica 10 veces por segundo hasta Ctrl+C (para `/cmd_vel`: el watchdog de la base frena si deja de llegar por 300 ms).
- El mensaje va en YAML: `{campo: valor, otro: {sub: valor}}`. Los campos que no se escriben quedan en 0.

### 7.4.1 Ejemplos con el robot

**Armar la base y avanzar** (con las ruedas en el aire la primera vez):

```
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 'armar'}"
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1}, angular: {z: 0.0}}"
# Ctrl+C: deja de publicar y en 300 ms la base frena sola (watchdog)
```

**Girar en el lugar** a 0,5 rad/s hacia la izquierda:

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{angular: {z: 0.5}}"
```

**Manejar con el teclado** (más cómodo):

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
# i = adelante, , = atrás, j/l = girar, k = parar, q/z = más/menos velocidad
```

**Brazo**: fijar el cero, armar y mover J2 a 45° (0,79 rad):

```
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 'z'}"
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 'armar'}"
ros2 topic pub --once /brazo/joint_cmd sensor_msgs/msg/JointState "{name: [j2], position:
    [0.79]}"
ros2 topic pub --once /brazo/joint_cmd sensor_msgs/msg/JointState "{name: [j1, j2, j3], position:
     [1.57, 1.2, -0.3]}"
```

**Pinza**: cerrar al 60 % y girar 45°:

```
ros2 topic pub --once /pinza/cierre std_msgs/msg/Float32 "{data: 60.0}"
ros2 topic pub --once /pinza/giro std_msgs/msg/Float32 "{data: 45.0}"
```

**Mandar el brazo a un punto** (con `brazo_ik` corriendo): el ejemplo de la E4 tiene que dar $q = (0°, 45°, 0°)$. Ojo: $q_1 = 0$ está fuera del rango de J1, así que se usa un punto a la izquierda:

```
ros2 run brazo_ik nodo_ik &
ros2 topic echo /brazo/ik_estado --field data &
ros2 topic pub --once /brazo/objetivo geometry_msgs/msg/PointStamped \
  "{header: {frame_id: brazo_base}, point: {x: 0.0, y: 0.36832, z: 0.09103}}"
# /brazo/ik_estado: "ok q=[90.0, 45.0, 0.0]°"
```

**Detener todo**:

```
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 's'}"
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 's'}"
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 'desarmar'}"
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 'desarmar'}"
```

## 7.5 Los tipos de mensaje que usa el robot

| Tipo | Tópicos | Campos que importan |
|---|---|---|
| geometry_msgs/Twist | /cmd_vel | linear.x [m/s], angular.z [rad/s] |
| nav_msgs/Odometry | /odom | pose.pose.position.x/y, pose.pose.orientation (cuaternión), twist.twist |
| sensor_msgs/Imu | /imu/data_raw | linear_acceleration [m/s²], angular_velocity [rad/s] |
| sensor_msgs/Range | /range/frente, izq, der | range [m]; +Inf = nada; NaN = sin eco |
| std_msgs/UInt8 | /bumpers | bit 0 frente, bit 1 izquierda, bit 2 derecha |
| sensor_msgs/BatteryState | /battery | voltage [V] |
| std_msgs/String | /base/estado, /base/comando, /brazo/estado… | data: texto |
| sensor_msgs/JointState | /brazo/joint_cmd, /brazo/joint_states | name (j1..j3), position [rad], velocity [rad/s] |
| std_msgs/Float32 | /pinza/cierre, /pinza/giro | data: % o grados |
| geometry_msgs/PointStamped | /limon/posicion, /brazo/objetivo | header.frame_id, point.x/y/z [m] |
| vision_msgs/Detection2DArray | /yolo/detecciones | cajas (centro, tamaño) y confianza |
| sensor_msgs/NavSatFix | /gps/fix | latitude, longitude |

## 7.6 Escribir un nodo propio

Ejemplo completo: un nodo que escucha /range/frente y, si el robot está armado, avanza despacio hasta que haya algo a menos de 40 cm. Sirve como plantilla.

```
# ros2_ws/src/mi_paquete/mi_paquete/avanzar_hasta.py
import math
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import Range


class AvanzarHasta(Node):
    def __init__(self):
        super().__init__('avanzar_hasta')                       # nombre del nodo
        self.declare_parameter('distancia', 0.40)               # parámetro con valor por defecto
     [m]
        self.declare_parameter('velocidad', 0.08)               # [m/s]
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)  # tópico, cola de 10
        self.create_subscription(Range, '/range/frente', self.al_rango, 10)
        self.create_timer(0.1, self.paso)                       # 10 Hz (el watchdog de la base
    es de 300 ms)
        self.rango = None
```

```
    def al_rango(self, msg):                                    # se llama con cada mensaje que
    llega
        self.rango = msg.range

    def paso(self):
        cmd = Twist()                                           # todo en 0 = frenar
        d = self.get_parameter('distancia').value
        if self.rango is not None and not math.isnan(self.rango) and self.rango > d:
            cmd.linear.x = self.get_parameter('velocidad').value
        self.pub.publish(cmd)


def main():
    rclpy.init()
    nodo = AvanzarHasta()
    try:
        rclpy.spin(nodo)                                        # atiende callbacks hasta Ctrl+C
    except KeyboardInterrupt:
        pass
    nodo.pub.publish(Twist())                                   # frenar al salir
    nodo.destroy_node()
    rclpy.try_shutdown()
```

Para registrarlo, en el `setup.py` del paquete:

```
entry_points={'console_scripts': ['avanzar_hasta = mi_paquete.avanzar_hasta:main']},
```

Y para correrlo:

```
cd ~/recolector_de_frutas/ros2_ws && colcon build --symlink-install --packages-select mi_paquete
source install/setup.bash
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 'armar'}"
ros2 run mi_paquete avanzar_hasta --ros-args -p distancia:=0.5 -p velocidad:=0.05
```

**Cómo están hechos los nodos del robot**

`mision` y `recorrido` separan la **lógica** (`fsm.py`, `recorrido.py`, sin ROS) del **nodo** (`nodo_mision.py`, que solo traduce tópicos a llamadas). Así la lógica se prueba con `pytest` sin ROS ni hardware. `brazo_ik` hace lo mismo con `cinematica.py`. Conviene seguir ese patrón en los nodos nuevos.

## 7.7 Un *launch* mínimo

```
# launch/demo.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='brazo_ik', executable='nodo_ik', parameters=[{'largo_herramienta': 0.075}])
    ,
        Node(package='tf2_ros', executable='static_transform_publisher',
             arguments=['--x', '-0.10', '--y', '0', '--z', '0.35',
                        '--frame-id', 'brazo_base', '--child-frame-id', 'stereo_camera_link']),
    ])
```

```
ros2 launch mi_paquete demo.launch.py
```

## 7.8 Marcos de coordenadas (TF) del robot

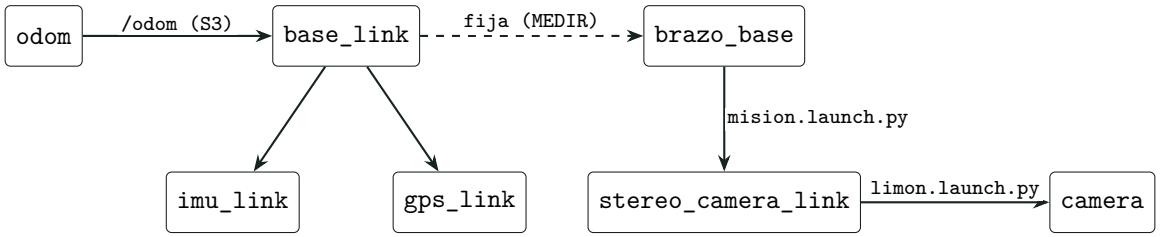



**Figura 7.2:** Árbol de marcos. `camera` es el marco óptico **rectificado** de la cámara izquierda ($x$ a la derecha, $y$ hacia abajo, $z$ hacia adelante). `/limon/posicion` viene en `camera`; `/brazo/objetivo` tiene que estar en `brazo_base`.

CAPÍTULO 8

# micro-ROS: cómo hablan la Pi y los ESP32

## 8.1 Qué es micro-ROS

Un ESP32 no puede correr ROS 2 completo (no tiene sistema operativo ni red DDS). **micro-ROS** es una versión reducida que corre en el microcontrolador y habla con un **agente** en la Pi por un enlace simple (acá, el USB serie). El agente traduce: para el resto de ROS 2, el ESP32 es un nodo más.

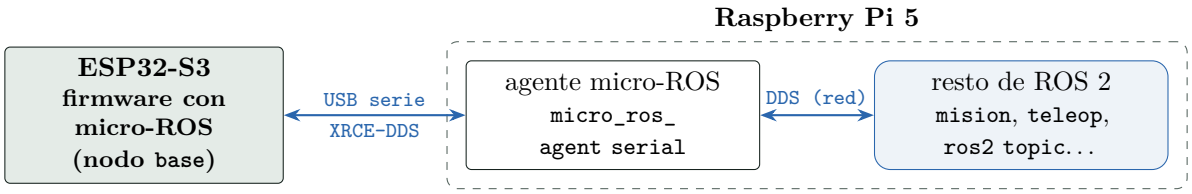



**Figura 8.1:** El agente es el puente. Hay **un agente por placa**, cada uno en su puerto USB.

### 8.1.1 Arrancar los agentes (en la Pi)

```
source /opt/ros/jazzy/setup.bash
source ~/microros_ws/install/local_setup.bash
# Base (ESP32-S3, USB nativo)
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/serial/by-id/usb-
    Espressif_USB_JTAG_serial_debug_unit_*-if00
# Brazo (WROOM con CH340) -- a 115200 baudios: a 921600 el CH340 pierde datos en la Pi
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/serial/by-id/usb-1a86_USB_Serial-if00-
    port0
```

**Flashear con el agente corriendo**

El agente tiene abierto el puerto USB: hay que detenerlo antes de flashear (`pio run -e ros -t upload`) y volver a arrancarlo después. Si la S3 no abre sesión, puede haber quedado en **modo descarga** (pasa si el puerto se abre con DTR/RTS bajos). Se saca con: `~/venvs/esptool/bin/esptool -port <S3> -before no_reset -after watchdog-reset read-mac`.

## 8.2 La sesión: conexión, hora y ping

1. Al arrancar, el ESP32 busca al agente. Mientras no lo encuentra, no publica nada.
2. Al conectarse, crea su nodo, sus publicadores y suscriptores, y **sincroniza la hora** con el agente (todos los mensajes llevan la hora de la Pi). Sin hora, no publica.
3. Cada **100 ms** hace un **ping** al agente. Si fallan 3 seguidos (∼1,5 s con los reintentos), considera la sesión perdida: **detiene los motores apenas lo detecta** y queda **desarmado**.

4. Vuelve a buscar al agente y, al reconectar, arranca una sesión nueva, otra vez desarmada.

`/<parte>/estado` incluye los contadores `enlace: pings_fallidos cortes`: sirven para ver si el USB anda bien. En la prueba con la Pi a plena carga hubo 0 cortes.

## 8.3 Armar y desarmar

**Qué es armar**

Cada ESP32 tiene un **seguro por sesión**. Al empezar cada sesión con el agente queda **DESARMADO**: ignora las órdenes de movimiento (`/cmd_vel` en la base; `/brazo/joint_cmd`, la pinza y los comandos que mueven en el brazo). Para habilitarlo se publica el texto `armar` en `/base/comando` o `/brazo/comando`.

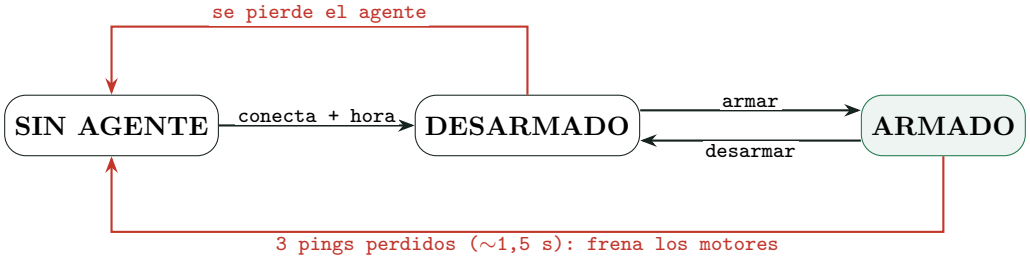



**Figura 8.2:** Estados de la sesión micro-ROS de cada ESP32.

- **Desarmado acepta**: detener (`s`), liberar (`l`), borrar fallas (`r`), cero (`z`), diagnóstico (`e`, `sens`), `cal`, `wd`.
- **Desarmado rechaza**: `/cmd_vel`, `v`, `w`, `d`, `m`, `j`, `home`, `/brazo/joint_cmd`, `pinza`, `giro`.

### 8.3.1 Por qué existe

Imaginá que el USB hace falso contacto y la placa se reconecta. Si un nodo seguía publicando "avanzá a 0,1 m/s" (por ejemplo, el teclado que quedó abierto), sin el seguro el robot arrancaría solo al volver la conexión. Con el seguro, después de cada reconexión **alguien tiene que volver a decir `armar` a propósito**.

### 8.3.2 Quién arma

- A mano: `ros2 topic pub -once /base/comando std_msgs/msg/String "{data: 'armar'}"`.
- `mision` arma el brazo en su estado INICIO, reintentando cada segundo hasta ver `ARMADO` en `/brazo/estado`. No lo vuelve a armar después de una reconexión: eso queda como falla.
- `recorrido` arma la base en su estado INICIO, igual.
- `mision` **desarma** el brazo si ve `SETA` en `/base/estado`.

### 8.3.3 Armar no es todo

Estar armado solo significa "acepto órdenes de esta sesión". Las demás protecciones siguen activas: en la base, la seta, el vuelco, los bumpers, la distancia y el watchdog; en el brazo, el cero, los límites,

el atasco. Por ejemplo, con la seta apretada la base puede estar armada pero no se mueve. Al soltar la seta hay que mandar `r` (rearmar la seguridad) además de que esté armada.

## 8.4 Watchdog de comandos

En la base, con micro-ROS el watchdog de comandos **arranca en 300 ms y no se puede apagar** (se puede cambiar entre 50 y 1000 ms con `wd`). Si `/cmd_vel` deja de llegar por más de ese tiempo, la base frena. Por eso `ros2 topic pub` para moverse se usa con `-r 10` (10 veces por segundo) y no con `-once`.

**Pendiente de medir o probar**

Medir con las ruedas en el piso la distancia real de frenado por el watchdog. Estimación (revisión de Codex): a 1 m/s, ~30 cm antes de empezar a frenar; en la simulación, 11–13 cm desde 0,2 m/s.

## 8.5 Tópicos de la base

| Tópico | Tipo | Sentido | Detalle |
|---|---|---|---|
| `/odom` | `nav_msgs/Odometry` | sale, 25 Hz | `odom` → `base_link` |
| `/imu/data_raw` | `sensor_msgs/Imu` | sale, 50 Hz | m/s² y rad/s, sin orientación |
| `/range/frente`, `/range/izq`, `/range/der` | `sensor_msgs/Range` | sale, 10 Hz | cada sensor mide cada 120 ms; +Inf = nada al alcance; NaN = sin eco |
| `/bumpers` | `std_msgs/UInt8` | sale, 10 Hz | bits 0..2: frente, izquierda, derecha |
| `/battery` | `sensor_msgs/BatteryState` | sale, 1 Hz | tensión |
| `/base/estado` | `std_msgs/String` | sale, 2 Hz | armado, modo, seguridad, bloqueos, IMU, fallas, enlace |
| `/base/respuesta` | `std_msgs/String` | sale | lo que mostraría el monitor serie |
| `/cmd_vel` | `geometry_msgs/Twist` | entra | `linear.x` [m/s], `angular.z` [rad/s] |
| `/base/comando` | `std_msgs/String` | entra | cualquier comando del monitor, más `armar` y `desarmar` |

## 8.6 Tópicos del brazo

| Tópico | Tipo | Sentido | Detalle |
|---|---|---|---|
| `/brazo/joint_states` | `sensor_msgs/JointState` | sale, 20 Hz | `j1..j3`: posición [rad], velocidad [rad/s] |
| `/brazo/estado` | `std_msgs/String` | sale, 2 Hz | armado; modo, cero, duty, falla y `llego` de cada articulación |
| `/brazo/respuesta` | `std_msgs/String` | sale | lo que mostraría el monitor serie |
| `/brazo/joint_cmd` | `sensor_msgs/JointState` | entra | posiciones [rad] con nombres `j1..j3` (o las 3 en orden). Se valida todo: se aplica todo o nada |
| `/pinza/cierre` | `std_msgs/Float32` | entra | 0 = abierta, 100 = cerrada |

| Tópico | Tipo | Sentido | Detalle |
|---|---|---|---|
| /pinza/giro | std_msgs/Float32 | entra | grados, −90..90 |
| /brazo/comando | std_msgs/String | entra | cualquier comando del monitor (z, home 2, s, r…), más armar y desarmar |

Si se pierde el agente, J1–J3 se detienen y el brazo queda desarmado; **los servos quedan donde están**, para no soltar el limón.

CAPÍTULO 9

# Visión: encontrar el limón en 3D

La visión vive en un repositorio aparte, `HecVelaz/vision_estereo` (en la Pi: `~/vision_estereo`). Toda la cadena corre en la Raspberry Pi.

## 9.1 La cadena completa

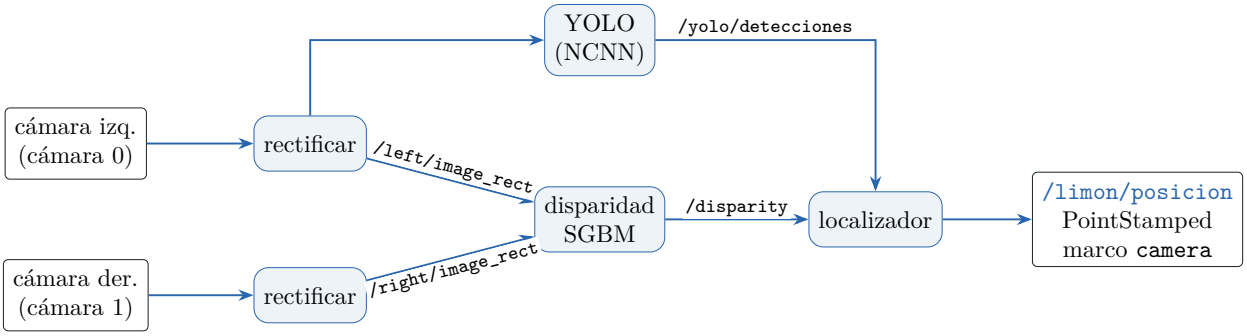



**Figura 9.1:** De las dos imágenes a la posición del limón. YOLO corre solo sobre la imagen izquierda rectificada; el estéreo da la profundidad de cada píxel.

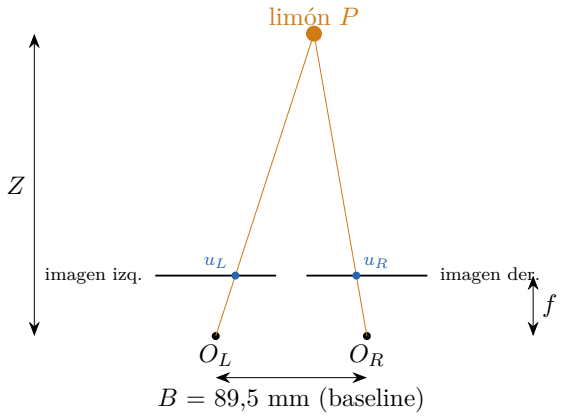
## 9.2 Las cámaras

- Dos **Raspberry Pi Camera Module 3 NoIR** (sensor IMX708), por CSI. **La cámara 0 de libcamera es la izquierda** (verificado con la paralaje de un objeto cercano).
- Sensor completo 2304:1296 escalado a 640 × 480, **foco fijo** (izquierda 2,90, derecha 2,15 dioptrías), 30 FPS (10 FPS dentro de la cadena del limón, para dejar CPU).
- Todo sale de un solo archivo: `config/camaras.yaml` (índices, modo, foco, FPS y carpeta de calibración).
- **NoIR** significa sin filtro infrarrojo: al sol los colores salen rosados y el detector puede fallar. Pendiente de probar al aire libre.
- Las dos cámaras **no están sincronizadas**: cada una toma la foto en su momento. Con el robot en movimiento la disparidad sale mal. Por eso **se localiza con el robot quieto**.

## 9.3 Cómo funciona el estéreo

### 9.3.1 La idea

Mirá un dedo con un ojo cerrado y después con el otro: el dedo "salta" respecto del fondo, y salta más cuanto más cerca está. Las dos cámaras hacen lo mismo. Si un punto aparece en la columna $u_L$ de la imagen izquierda y en $u_R$ de la derecha, la **disparidad** es $d = u_L - u_R$ (en píxeles).




$$Z = \frac{f\,B}{d}, \quad d = u_L - u_R$$

$$X = \frac{(u - c_x)\,Z}{f}, \quad Y = \frac{(v - c_y)\,Z}{f}$$

$f = 654{,}3$ px, $B = 0{,}0895$ m $\Rightarrow fB = 58{,}6$ px · m

$Z = 0{,}40$ m $\Rightarrow d = 146$ px; $Z = 0{,}90$ m $\Rightarrow d = 65$ px

Error: $\Delta Z \approx \frac{Z^2}{fB}\,\Delta d$; con $\Delta d = 0{,}5$ px a 0,9 m, $\Delta Z \approx 7$ mm

**Figura 9.2:** Triangulación estéreo. El error crece con el cuadrado de la distancia: por eso el limón se mira de cerca.

### 9.3.2 Calibración y rectificación

Para usar esas fórmulas hacen falta $f$, $B$, el centro óptico $(c_x, c_y)$, la distorsión de cada lente y la rotación entre las dos cámaras. Se miden con una **calibración estéreo**: se fotografía un tablero de ajedrez ($9 \times 6$ esquinas, cuadros de 25 mm) en unas 30 posiciones y OpenCV ajusta todos los parámetros.

Con eso se **rectifican** las imágenes: se deforman para que las dos queden como si las cámaras fueran perfectamente paralelas. Entonces un punto aparece **en la misma fila** de las dos imágenes, y buscar su pareja es buscar solo a lo largo de esa fila.

| **Calibración en uso** | `config/calibracion_soporte_2/` (soporte rígido, 01/10) |
|---|---|
| Pares de imágenes | 34 |
| Error RMS estéreo | 0,36 px |
| Error epipolar medio (p95) | 0,08 px (0,21 px) |
| Baseline | 89,5 mm (con regla: 90 mm) |
| Focal rectificada | 654,3 px |
| Ventana de trabajo | 0,38–0,96 m (disparidad 61–156 px) |

**La calibración se pierde si algo se mueve**

Vale solo para el mismo modo de sensor, resolución y foco, y **mientras las cámaras no se muevan entre sí**. Si el soporte se golpea o se cambia el foco, hay que recalibrar.

```
python3 ~/vision_estereo/herramientas/ajustar_foco.py                # foco por cámara (tablero
    a 55 cm)
python3 ~/vision_estereo/herramientas/calibrar_estereo.py capturar   # captura guiada (vista: /
    calibracion/vista)
python3 ~/vision_estereo/herramientas/calibrar_estereo.py calcular --salida config/calibracion_<
    nombre>
# después: apuntar "calibracion:" de config/camaras.yaml a la carpeta nueva
python3 ~/vision_estereo/herramientas/medir_profundidad_tablero.py   # control de escala
```

### 9.3.3 Disparidad: SGBM

El nodo de disparidad calcula $d$ para cada píxel. Hay dos algoritmos:

**BM** (*block matching*): rápido (∼10 Hz), más ruidoso, ∼500 puntos por limón.

**SGBM** (*semi-global*): más lento (∼2,5 Hz), mucho más limpio, ∼1000 puntos por limón. **Es el que se usa** (por defecto desde el 01/10; `ALGORITMO=0` vuelve a BM).

Solo se buscan disparidades entre 61 y 156 px (0,38 a 0,96 m). Eso tiene un costo: la **zona ciega**. Un punto en la columna $u$ de la imagen izquierda solo se puede emparejar si $u - d \geq 0$; con la ventana actual, el cuarto izquierdo de la imagen ($u <$ ∼163 px) **se detecta pero no se localiza**.

## 9.4 El detector YOLO

- Modelo **YOLO26n** (nano) con una sola clase, `limon`. Entrenado en Google Colab (`vision/entrenamiento_limon/`) con datasets de Roboflow.
- Exportado a **NCNN** para la CPU de la Pi: 23 FPS solo; 7–9 FPS dentro de la cadena completa.
- En la Pi: `~/vision_estereo/modelos/limon_yolo26n_ncnn_model` (fuera de git). El `.pt` está en Drive.
- Prueba (285 imágenes, 173 limones): precisión 0,953, recobrado 0,977, mAP50 0,978, mAP50-95 0,814. Son números optimistas: las imágenes de prueba son de las mismas fuentes que las de entrenamiento.
- Con un limón real en el laboratorio: confianza mediana 0,80.

**Clases de la guía**

La guía (B3) pide distinguir **cosechable / no cosechable / obstáculo u oclusión**. Hoy hay una sola clase.

## 9.5 El localizador: de la caja al punto 3D

YOLO da una caja; la disparidad da una profundidad por píxel. El problema es que dentro de la caja hay también hojas y ramas detrás del limón. El `localizador_node` (`yolo_ros/localizar.py`) hace:

1. Toma solo el **centro de la caja** (el 50 % central de cada lado): los bordes suelen ser fondo.
2. Convierte cada disparidad válida a profundidad $Z = fB/d$ y descarta las que están fuera de 0,15–2 m.
3. **Ordena las profundidades** y corta donde hay un salto de más de 5 cm: eso separa el limón del fondo.
4. Toma el **primer grupo** (el más cercano) que tenga al menos 15 puntos (los grupitos sueltos son ruido).
5. Del grupo usa solo los primeros 8 cm de profundidad (un limón mide ∼6 cm).
6. Calcula $X$, $Y$, $Z$ con los píxeles de ese grupo y publica `/limon/posicion`: el limón de mayor confianza que se pudo localizar.

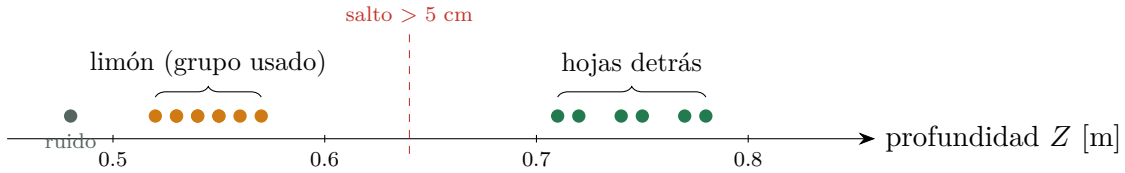



**Figura 9.3:** Corte por saltos de profundidad. El punto suelto más cercano no llega a 15 puntos y se descarta.

**Superficie, no centro**

El punto publicado es la **superficie del limón que ve la cámara**. La misión le suma el radio del limón (3 cm) a lo largo del rayo de la cámara para obtener el centro.

**Marco rectificado**

`/limon/posicion` está en el marco óptico **rectificado** de la cámara izquierda (`camera`): $x$ a la derecha, $y$ hacia abajo, $z$ hacia adelante. Difiere del marco físico de la cámara en unos pocos grados (la rotación de la rectificación). Como la TF cámara → brazo se mide contra la cinta, esa diferencia queda absorbida.

## 9.6 Resultados medidos

| **Cinta [mm]** | 400 | 550 | 700 | 900 |
|---|---|---|---|---|
| Tablero (estéreo) | 387,5 | 536,2 | 702,7 | 898,7 |
| Limón real, BM | 409,5 | 549,6 | 717,1 | 921,4 |
| Limón real, SGBM | 409,3 | 554,3 | 712,5 | 914,6 |

- Con SGBM en el eje: error de +4 a +15 mm, ruido de ±0,3–0,9 mm.
- Limón a los costados (6 puntos medidos con regla): error 3D máximo **17 mm**. La guía (B5) pide ≤ 35 mm: **cumple en el laboratorio**.
- Queda ∼+1 cm de desfase en $Z$: el origen de la cinta (frente de la lente, no el centro óptico) más ∼1 % de escala.

**Pendiente de medir o probar**

Prueba de campo (limón en un arbusto a 40–90 cm, `guia_limon.py -campo`) y prueba de las NoIR al sol.

## 9.7 Cómo se usa

```
# En la Pi: cámaras + rectificación + SGBM + YOLO + localizador
MODELO=~/vision_estereo/modelos/limon_yolo26n_ncnn_model CLASE=limon ~/vision_estereo/scripts/
    start_limon.sh

# En otra terminal:
ros2 topic echo /limon/posicion          # posición del limón [m], marco camera
ros2 topic hz /limon/posicion            # 2-4 Hz con un limón a la vista

# Ver las imágenes desde la notebook (web_video_server en el puerto 8080):
#   http://100.116.96.71:8080/stream?topic=/yolo/left/image_annotated
```

| Script | Qué hace |
|---|---|
| `start_cameras.sh` (alias `camaras`) | Solo las dos cámaras, a 30 FPS |
| `start_yolo_web.sh` (alias `yolo`) | Cámaras + YOLO + visor web |
| `start_estereo.sh` (alias `estereo`) | Estéreo completo con nube de puntos `/points2` |
| `start_limon.sh` | La cadena del limón (lo que usa el robot) |
| `probar_camaras.sh` (en `~/scripts`) | Diagnóstico de las cámaras |

**Si una cámara falla**

Síntomas de cinta CSI dañada: `Failed to write reg`, `stream on failed`, `Camera frontend has timed out`. Se aísla intercambiando las cintas, **siempre con la Pi apagada**.

CAPÍTULO 10

# Cinemática del brazo

## 10.1 El problema

La visión dice **dónde** está el limón: un punto $(x, y, z)$ en metros. El firmware del brazo necesita **ángulos**: $q_1$, $q_2$, $q_3$. Pasar de uno a otro es la **cinemática inversa** (IK). El camino contrario, de ángulos a punto, es la **cinemática directa**. Las dos están en `ros2_ws/src/brazo_ik/brazo_ik/cinematica.py`, portadas del cuaderno de la entrega R1 E4.

## 10.2 Cinemática directa

Con las medidas de la figura 3.4 (en mm):

1. **Cuatro barras**: el ángulo de entrada es $\gamma_4 = q_2 + q_3 - 90°$. Del cierre del mecanismo (balancines $L_{3,0}$, $L_{3,1}$ y acoplador $L_4$) sale el ángulo del antebrazo respecto de la prolongación de $L_2$:

$$\ell = \sqrt{L_2^2 + L_{3,0}^2 - 2L_2 L_{3,0}\cos\gamma_4}, \qquad \alpha_4 = \operatorname{atan2}(L_{3,0}\sin\gamma_4,\ L_2 - L_{3,0}\cos\gamma_4)\,,$$

$$\beta_4 = \arccos\frac{\ell^2 + L_{3,1}^2 - L_4^2}{2L_{3,1}\,\ell}$$

$$q_{3O} = \alpha_4 - \beta_4$$

   Si el argumento del arccos se sale de $[-1, 1]$, el mecanismo no cierra: esa configuración no existe.

2. **Ángulo de transmisión** $\mu$ (entre $L_{3,1}$ y $L_4$): si se acerca a 0° o 180°, el mecanismo está cerca de un punto muerto y no transmite fuerza. Se exige $15° \le \mu \le 165°$.

3. **Muñeca** $P_3$: con $\theta = q_2 + q_{3O}$,

$$r = L_2\cos q_2 + L_3\cos\theta, \qquad x = r\cos q_1, \quad y = r\sin q_1, \quad z = H_1 + L_2\sin q_2 + L_3\sin\theta$$

**Ejemplo de la E4**: $q = (0°, 45°, 0°)$ da la muñeca en (368,32; 0; 91,03) mm, con $q_{3O} = -113{,}5°$. Con $q_1 = 90°$, el mismo brazo apunta a la izquierda: (0; 368,32; 91,03).

## 10.3 Cinemática inversa, paso a paso

Dado el objetivo de la muñeca $(x, y, z)$:

1. $q_1$: la dirección horizontal, $q_1 = \operatorname{atan2}(y, x)$, llevada al rango $[45°, 225°]$ (atan2 da $(-180°, 180°]$; un objetivo atrás a la derecha da $-150°$, que es 210°).

2. **Problema plano**: $r = \sqrt{x^2 + y^2}$, $z' = z - H_1$. Queda un brazo de dos eslabones $L_2$, $L_3$ en el plano $(r, z')$.

3. **Codo** $q_{3O}$ por el teorema del coseno:

$$\cos q_{3O} = \frac{r^2 + z'^2 - L_2^2 - L_3^2}{2L_2L_3}$$

Si $|\cos q_{3O}| > 1$: el punto está demasiado lejos ($> L_2 + L_3 = 679{,}7$ mm del hombro) o demasiado cerca. Hay dos soluciones, $\pm q_{3O}$ ("codo arriba" y "codo abajo"): se prueban las dos.

4. **Hombro** $q_2$:

$$q_2 = \operatorname{atan2}(z', r) - \operatorname{atan2}(L_3 \sin q_{3O},\ L_2 + L_3 \cos q_{3O})$$

5. **Invertir el cuatro barras**: de $q_{3O}$ se obtienen hasta dos $\gamma_4$ candidatos, y $q_3 = \gamma_4 - q_2 + 90°$.
6. **Verificar con la directa**: cada candidato se pasa por la cinemática directa; si no vuelve al objetivo (a menos de $10^{-4}$ mm), pertenece a la otra rama de ensamble y se descarta.
7. **Filtros**: límites de $q_1$, $q_2$, $q_3$; rama física $\gamma_4 \in (-180°, 0°)$ (la barra roja no cruza la verde); ángulo de transmisión entre 15° y 165°.
8. De las soluciones que quedan, se elige la de menor $|q_3|$.

Si no queda ninguna, la función devuelve los **motivos** ("q2 fuera de rango", "demasiado lejos", "cerca de punto muerto"…), que el nodo publica en `/brazo/ik_estado`.

## 10.4 Del punto de agarre a la muñeca

El brazo mantiene la herramienta horizontal y apuntando en la dirección de $q_1$. Entonces el **punto de agarre** (TCP, entre los dedos) está siempre a una distancia horizontal fija, `largo_herramienta`, más afuera que la muñeca, en el mismo plano vertical:

$$r_{\text{muñeca}} = r_{\text{TCP}} - \texttt{largo_herramienta}, \qquad (x, y)_{\text{muñeca}} = (x, y)_{\text{TCP}} \frac{r_{\text{muñeca}}}{r_{\text{TCP}}}, \qquad z_{\text{muñeca}} = z_{\text{TCP}}$$

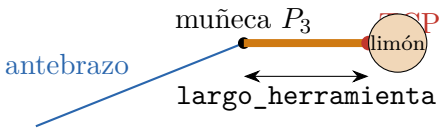



**Figura 10.1:** `/brazo/objetivo` es el TCP. `brazo_ik` lo pasa a la muñeca antes de resolver.

**Pendiente de medir o probar**

`largo_herramienta` vale 0 por defecto (el objetivo es la muñeca). En la simulación, 0,075 m. Hay que **medirlo en el robot real** y pasarle el mismo valor a `brazo_ik` y a `mision`. Si el centro del agarre no queda a la altura de la muñeca, falta además un desfase vertical (hoy no contemplado).

## 10.5 El nodo `brazo_ik`

| Tópico | Tipo | Qué |
|---|---|---|
| `/brazo/objetivo` (entra) | `PointStamped` | TCP en metros, marco `brazo_base` (otro marco se rechaza) |
| `/brazo/joint_cmd` (sale) | `JointState` | `j1..j3` en radianes, **solo si es alcanzable** |
| `/brazo/ik_estado` (sale) | `String` | `ok q=[...]° mu=...` o `inalcanzable: <motivos>` |

Parámetros: `marco` (`brazo_base`) y `largo_herramienta` (m).

```
ros2 run brazo_ik nodo_ik --ros-args -p largo_herramienta:=0.0
ros2 topic echo /brazo/ik_estado --field data
ros2 topic pub --once /brazo/objetivo geometry_msgs/msg/PointStamped \
  "{header: {frame_id: brazo_base}, point: {x: 0.0, y: 0.36832, z: 0.09103}}"
# -> ok q=[90.0, 45.0, 0.0]° mu=65.1° err=4.77e-13 mm
```

**Límites en dos lugares**

La geometría y los límites de `cinematica.py` **tienen que coincidir** con `firmware/brazo/include/brazo_config.h` y con el URDF de la simulación. Si se cambia uno, cambiar los tres.

Pruebas: `colcon test -packages-select brazo_ik` (ejemplo numérico de la E4, 2000 idas y vueltas directa → inversa → directa, casos inalcanzables).

CAPÍTULO 11

# Misión y recorrido autónomo

El paquete `mision` (`ros2_ws/src/mision/`) tiene dos máquinas de estados que trabajan juntas:

**`fsm.py` — la misión** Maneja el **brazo**: espera un limón estable, lo manda a agarrar, lo retira y lo suelta.

**`recorrido.py` — el recorrido** Maneja la **base**: avanza por el pasillo, frena cuando ve un limón, se arrima si está fuera de alcance, esquiva obstáculos. Se activa con el parámetro `autonomo:=true`.

Las dos son Python puro, sin ROS (se prueban con `colcon test`: 30 pruebas de la misión). `nodo_mision.py` es el nodo que las conecta con los tópicos.

## 11.1 La misión (brazo)

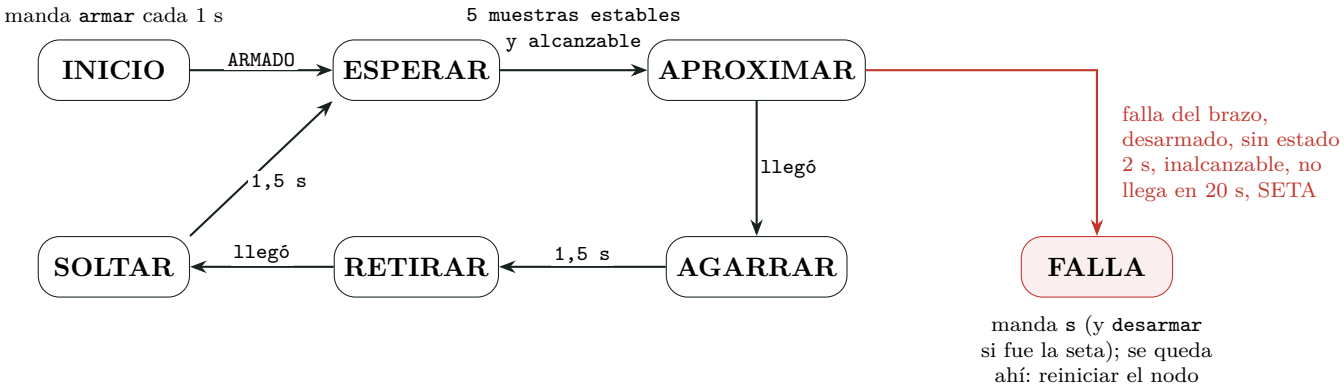



**Figura 11.1:** Máquina de estados de la misión. Cualquier estado con el brazo moviéndose puede ir a FALLA.

### 11.1.1 Cada estado

**INICIO** Manda `armar` al brazo cada segundo hasta ver `ARMADO` en `/brazo/estado`.

**ESPERAR** Junta posiciones de `/limon/posicion`, pero **solo** si el brazo está armado, con cero, sin falla, quieto y con un `/brazo/estado` de hace menos de 2 s, y (con el recorrido) la base quieta. Cada muestra se pasa al marco del brazo con TF y se le suma el radio del limón (3 cm) a lo largo del rayo de la cámara. Descarta muestras de imágenes de más de 1 s y las repetidas. Cuando hay **5 seguidas a menos de 2 cm del promedio** (en una ventana de 3 s), pregunta si el promedio es alcanzable (la misma cuenta que `brazo_ik`). Si sí, pasa a APROXIMAR; si no, avisa "fuera de alcance" al recorrido.

**APROXIMAR** Publica el punto en `/brazo/objetivo`. Espera a que el brazo **llegue**.

**AGARRAR** Cierra la pinza (`/pinza/cierre` = `cierre`, 100 % por defecto: **MEDIR** sin aplastar) y espera 1,5 s.

**RETIRAR** Manda el brazo a la pose de reposo (TCP en (0,068; 0,253; 0,360) m, $q = (75°, 110°, -30°)$: no tapa las cámaras ni toca el mástil).

**SOLTAR** Abre la pinza, espera 1,5 s y vuelve a ESPERAR.

**FALLA** Manda `s` (detiene J1–J3; la pinza queda como está) y se queda ahí. Hay que reiniciar el nodo. Al cerrar el nodo (Ctrl+C) también manda `s`.

### 11.1.2 Qué significa "llegó"

No alcanza con que el firmware diga `llego`: podría ser un estado viejo de un movimiento anterior. "Llegó" exige las tres cosas a la vez:

1. `/brazo/joint_states` a menos de 1° de los ángulos que calculó `brazo_ik` **para este objetivo** (los de `/brazo/joint_cmd`);
2. velocidad de las tres articulaciones $< 0{,}02$ rad/s (quieto);
3. `llego` en las tres en `/brazo/estado`.

### 11.1.3 La seta y la misión

La WROOM no ve la seta (está conectada a la S3). Si la seta corta los 12 V y la WROOM sigue armada, al soltar la seta el brazo arrancaría de golpe con el error acumulado, mientras J2 y J3 caían. Por eso la misión escucha `/base/estado`: si aparece `SETA`, manda `desarmar` al brazo y pasa a FALLA.

## 11.2 El recorrido (base)

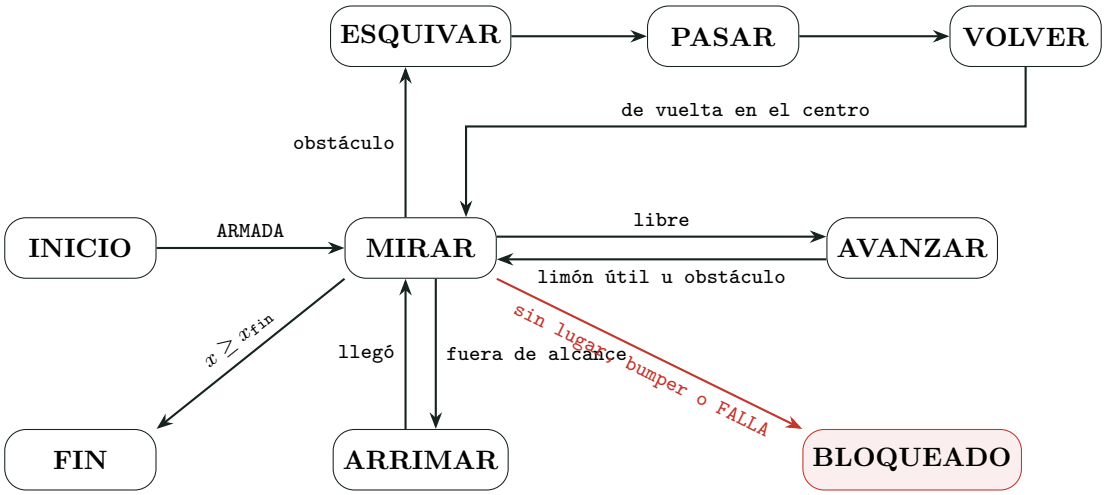



**Figura 11.2:** Máquina de estados del recorrido. La base **solo se mueve con la misión en ESPERAR** (brazo en reposo).

**INICIO** Frena y manda `armar` a la base cada segundo hasta verla ARMADA.

**AVANZAR** Va a 0,10 m/s por el centro del pasillo ($y = 0$), corrigiendo el rumbo. Frena (pasa a MIRAR) si ve un limón útil, si hay un obstáculo a menos de 0,30 m delante o al llegar al fin de la hilera (7,3 m).

**MIRAR** Quieto. Si desde que frenó vio un limón útil, le da 3 s a la misión; si no vio nada, sigue a los 0,5 s. Después decide: arrimar, retroceder por un limón que quedó atrás, avanzar, esquivar o terminar.

**ARRIMAR** Se mueve a lo largo del pasillo hasta una $x$ objetivo, con la odometría. Como mucho 3 arrimes seguidos sin cosechar; después sigue de largo.

**ESQUIVAR, PASAR, VOLVER** Se corre a un carril lateral ($y = \pm 0{,}25$ m) del lado con lugar, pasa el obstáculo y vuelve al centro, derecho.

**BLOQUEADO** No hay lugar para esquivar, un bumper está apretado o la misión está en FALLA: frena y reintenta.

### 11.2.1 Cuándo frena por un limón

YOLO da cajas, no distancias. Pero un limón mide unos 6 cm, así que el **ancho de la caja** da una distancia aproximada:

$$Z \approx \frac{f \cdot 0{,}06 \text{ m}}{\text{ancho [px]}}$$

Un limón es **útil** si esa $Z$ está en la ventana del estéreo y si está a la derecha de la zona ciega ($u \geq u_{\text{mín}}$). Un limón en la zona ciega ($u < u_{\text{mín}}$) con la cámara mirando de costado es un limón que **ya quedó atrás**: el recorrido retrocede de a 10 cm (como mucho 40 cm por tramo, porque atrás no hay sensores) hasta verlo en la zona útil.

**Parámetros del recorrido para el robot real**

`ParRecorrido` en `recorrido.py` usa la calibración vieja de la simulación: $f = 1076{,}5$ px, $u_{\text{mín}} = 271$ px, ventana 0,40–0,95 m. Con las cámaras reales (calibración del soporte) son $f = 654{,}3$ px y $u_{\text{mín}} \approx 163$ px. Hay que pasarlos por parámetro (`f_px`, `u_min`, `z_min`, `z_max`) al probar en el robot.

### 11.2.2 Cuánto avanzar para alcanzar el limón

Esta es la respuesta a "¿cómo sabe la rueda hasta dónde moverse?". La rueda **no recibe las coordenadas del limón**. Pasa esto:

1. La misión tiene el limón en el marco del brazo, $p = (p_x, p_y, p_z)$, y la cinemática inversa dice que no se alcanza.
2. El recorrido prueba desplazamientos $\Delta x$ a lo largo del pasillo, de $-0{,}60$ a $+0{,}60$ m de a 1 cm. Mover la base $\Delta x$ hacia adelante equivale a que el limón quede en $(p_x - \Delta x,\ p_y,\ p_z)$ respecto del brazo.
3. Se queda con los $\Delta x$ para los que el limón es alcanzable **con margen** (también corrido $\pm 2$ cm), y de esos elige el **más chico**, para que el limón siga a la vista.
4. Pasa a ARRIMAR con $x_{\text{obj}} = x_{\text{actual}} + \Delta x$.
5. En ARRIMAR manda `/cmd_vel` lento, mira `/odom` (cuánto avanzaron las ruedas) y frena al llegar a $x_{\text{obj}}$ ($\pm 1$ cm).
6. Vuelve a MIRAR: el estéreo vuelve a medir y ahora la misión lo puede agarrar.

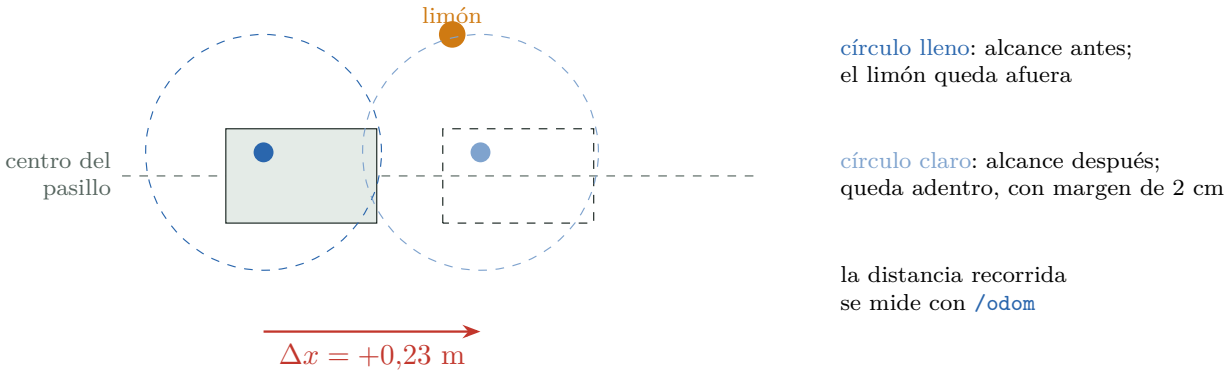



**Figura 11.3:** Arrimarse: el robot solo se mueve a lo largo del pasillo, lo justo para que el limón entre en el alcance del brazo.

### 11.2.3 Rumbo: ruedas para avanzar, giróscopo para girar

La posición que usa el recorrido combina:

- el **avance** medido por las ruedas (`/odom`);
- el **rumbo** integrado del giróscopo (`/imu/data_raw`, eje $z$).

En los giros cerrados las ruedas patinan: en la simulación, solo con la odometría el robot quedaba 13 cm corrido sin enterarse. El giróscopo no patina. En el robot real hay que calibrar su sesgo en reposo (`cal` de la base) y **arrancar en el centro del pasillo, mirando a lo largo** (el recorrido supone que $y = 0$ es el centro).

### 11.2.4 Esquivar

Los tres HC-SR04 miran al frente (0° y ±15°). Con cada eco, y suponiendo que puede venir de cualquier punto del cono de ±7,5°, el recorrido calcula qué parte del pasillo ocupa el obstáculo. Si hay lugar en un costado (ancho del robot + 5 cm de cada lado), se corre a ese carril, avanza el largo supuesto del obstáculo (0,30 m) más su propio largo y vuelve al centro.

## 11.3 Cómo se lanza

```
# En la Pi, con el agente del brazo y la visión (start_limon.sh) corriendo:
ros2 launch mision mision.launch.py x:=-0.10 y:=0 z:=0.35 roll:=0 pitch:=0 yaw:=0   # TF medida (
    MEDIR)
ros2 topic echo /mision/estado --field data

# Con el recorrido autónomo (también el agente de la base), parámetros del nodo:
ros2 run mision nodo_mision --ros-args -p autonomo:=true -p largo_herramienta:=0.075 \
  -p f_px:=654.3 -p u_min:=163.0 -p z_min:=0.40 -p z_max:=0.95
```

**La TF es obligatoria**

`mision.launch.py` **no arranca** sin los 6 argumentos de la TF `brazo_base` → `stereo_camera_link` (x, y, z, roll, pitch, yaw), para no mover el brazo con una TF inventada. Se miden con el soporte rígido de las cámaras ya montado: x, y, z con cinta; los ángulos se ajustan hasta que un limón medido a mano dé el mismo punto en `brazo_base`.

| Parámetro | Por defecto | Qué es |
|---|---|---|
| `n_muestras` | 5 | posiciones seguidas que tienen que coincidir |
| `tol_muestras` | 0,02 m | cada muestra a menos de esto del promedio |
| `ventana` | 3 s | muestras más viejas se descartan |
| `t_mov_max` | 20 s | tiempo máximo para llegar |
| `t_pinza` | 1,5 s | para cerrar o abrir |
| `tol_q`, `tol_v` | 1°, 0,02 rad/s | para considerar que llegó y está quieto |
| `cierre` | 100 % | cierre para agarrar (**MEDIR**) |
| `reposo` | (0,068; 0,253; 0,360) m | TCP de reposo |
| `largo_herramienta` | 0 m | muñeca → TCP (**MEDIR**; el mismo que `brazo_ik`) |
| `radio_limon` | 0,03 m | se suma a la superficie que da la visión |
| `edad_max_imagen` | 1 s | detecciones más viejas se ignoran |
| `autonomo` | `false` | activa el recorrido |

CAPÍTULO 12

# Simulación en Gazebo

El paquete `simulacion` (`ros2_ws/src/simulacion/`) arma en **Gazebo Harmonic** una hilera de limoneros y el robot, con **los mismos tópicos y marcos que los ESP32**. Así `mision`, `brazo_ik` y la visión corren sin cambios, sin hardware.

## 12.1 Qué hay en el mundo

- Pasillo a lo largo de $+x$, robot en el origen, **4 limoneros por lado** en $x = 1{,}5$ / 3 / 4,5 / 6 m, $y = \pm 0{,}70$ m, con las medidas de R1.
- **12 limones cosechables** (3 por árbol de la fila izquierda).
- 2 obstáculos en el pasillo (una caja y un balde) que obligan a esquivar.
- **Marcas de parada** blancas en $x = 1{,}19$ / 2,70 / 4,17 / 5,69 m: frenando con el eje de las ruedas sobre una marca, en el centro del pasillo, los 3 limones de ese árbol quedan a la vista y al alcance.

## 12.2 Quién imita a quién

| Nodo de la simulación | Reemplaza a | Detalle |
|---|---|---|
| `base_sim` | firmware de la S3 | misma seguridad (`SeguridadBase` portada), arranca DESARMADA |
| `brazo_sim` | firmware de la WROOM | mismos tópicos, perfil trapezoidal, arranca DESARMADO |
| `limones_sim` | visión | publica `/limon/posicion` solo con el robot quieto y el limón en la zona útil |
| 3 `gpu_lidar` | HC-SR04 | 5 rayos en un abanico de 15° cada uno |
| 3 planchas con resorte | bumpers | apretado = hundida más de 3 mm |

## 12.3 Cómo se usa

```
cd ~/Documentos/recolector_de_frutas/ros2_ws && source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select simulacion && source install/setup.bash
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST          # ¡en CADA terminal! (aislar de los ESP32
     reales)

ros2 launch simulacion sim.launch.py                    # robot y hilera (gui:=false: sin ventana
    )
ros2 launch simulacion sim.launch.py mision:=true       # + misión (arrimar con el teclado a una
    marca)
ros2 launch simulacion sim.launch.py mision:=true autonomo:=true   # cosecha sola toda la hilera

# Manejar a mano (otra terminal, con el export):
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: armar}"
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -p repeat_rate:=10.0
```

**repeat_rate**

Sin `repeat_rate:=10.0`, el teclado publica solo al tocar una tecla y el watchdog de 300 ms frena la base entre tecla y tecla.

**Aislar la simulación**

Sin `ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST`, la simulación ve por la red los ESP32 reales conectados a la Pi: el 28/09 la misión simulada leyó el estado del brazo real y le mandó comandos.

Si algo queda colgado después de cortar:

```
pkill -9 -f "[g]z sim"
pkill -9 -f "[b]razo_sim|[b]ase_sim|[l]imones_sim|[p]arameter_bridge"
```

## 12.4 Resultados

- Con `mision:=true autonomo:=true`: recorre la hilera hasta $x = 7{,}32$ m en ∼9 min simulados y cosecha **12 de 12** limones, esquivando la caja por la derecha y el balde por la izquierda, sin tocarlos.
- El watchdog frena la base a los 300 ms (11–13 cm desde 0,2 m/s con la rampa).
- Con la cadena de visión real (`vision:=yolo`): el estéreo localiza, pero YOLO no reconoce los limones sintéticos. No reentrenar con imágenes de Gazebo: arruinaría el modelo para los limones reales.

**Trucos de la física**

Las masas del brazo van al 5 % (si no, los impulsos del brazo deslizaban la base con las ruedas frenadas), el limón agarrado se lleva con `set_pose` y `limones_sim` usa la pose real de Gazebo, no la odometría. Son cosas de la simulación: en el robot real no aplican.

CAPÍTULO 13

# Puesta en marcha, paso a paso

Este capítulo es la receta para encender y probar el robot real, en orden. Cada paso se apoya en el anterior: no saltear pasos. **Hasta hoy (14/10) nada de esto se probó con los periféricos reales**: las placas solo se probaron solas.

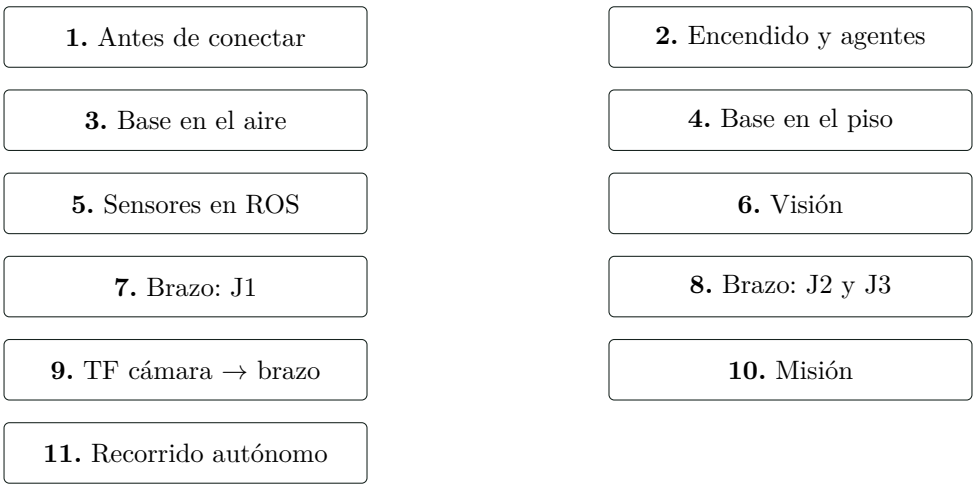



**Figura 13.1:** Orden de las pruebas.

## 13.1 Paso 0: antes de conectar

- ☐ **GND común**: un cable del GND de la S3 y otro del de la WROOM a la bornera de GND del bus de motores. Sin eso, los PWM no tienen referencia en los IBT-2. Los positivos de la LiPo y del power bank nunca se tocan.
- ☐ **Divisor de batería** 100 k / 30 k armado y medido **con la lógica apagada** antes de conectar la LiPo.
- ☐ **Fusible** de 20 A pegado al XT60. Cable del bus de al menos 14 AWG.
- ☐ **Buck 3** ajustado a 6 V sin carga, con el multímetro, antes de conectar los servos.
- ☐ **Seta**: verificar con el multímetro qué contactos son NC antes de cablearla.
- ☐ Pull-down de 10 kΩ en cada EN (S3 GPIO 38 y WROOM GPIO 23).
- ☐ Divisor 1 k / 2 k en cada ECHO de los HC-SR04.
- ☐ **Encoders de tracción**: medir sin alimentar la resistencia de cada salida a Vcc (debería dar ≈ 10 kΩ). Con los 20 kΩ a GND, girando despacio, el alto no tiene que pasar de 3,3 V.
- ☐ **Encoders del brazo** a 3,3 V (no a 5 V). Sin girar, A y B en 0 o 3,3 V, nunca a mitad; si no, falta el pull-up (10 kΩ a 3V3).
- ☐ **IBT-2 a 3,3 V**: leer el integrado de entrada de los 5 módulos. Con 74HC244 a 5 V el alto mínimo es 3,5 V: los 3,3 V del ESP32 quedan fuera de hoja. Probar cada uno a 20 kHz.
- ☐ **LEDs de la Freenove**: medir si cargan el GPIO 1 (lectura de batería), los encoders o el I²C.

- ☐ Cables a las borneras con cobre pelado, **sin estañar**.
- ☐ Brazo en reposo **apoyado en un tope** (J2 y J3 caen sin corriente).

## 13.2 Paso 1: encendido y agentes

1. Selector en **OFF**. Enchufar el power bank a la Pi y esperar que arranque.
2. Conectar por SSH: `ssh r2d2@100.116.96.71`.
3. Verificar alimentación: `vcgencmd get_throttled` tiene que dar `0x0`.
4. Verificar que la Pi tenga el repo y los binarios al día (ver `/pi-sync` en el capítulo 15).
5. Arrancar los dos agentes, cada uno en su terminal (sección 8.3).
6. Comprobar las tasas:

```
ros2 topic hz /odom                      # 25 Hz
ros2 topic hz /brazo/joint_states        # 20 Hz
ros2 topic echo /base/estado --field data
ros2 topic echo /brazo/estado --field data
```

7. Enchufar la LiPo (XT60) con el selector en OFF. Mirar el voltímetro.

## 13.3 Paso 2: base con las ruedas en el aire

Con la base sobre un soporte, ruedas sin tocar nada. Mejor en modo serie (firmware `esp32s3`, monitor abierto), o por `/base/comando` con micro-ROS (la respuesta sale en `/base/respuesta`).

1. `sens`: ver que respondan la IMU, las distancias, los bumpers, la seta y la batería. Comparar la tensión con el tester.
2. Selector en ON. `r` (rearmar la seguridad) y, con micro-ROS, `armar`.
3. `d 25 0` repetido (rueda izquierda), después `d 0 25` (derecha). Cada rueda tiene que girar **hacia adelante** y `e` mostrar rpm positivas.
   - Gira al revés → cambiar `MOTOR_INVERTIDO` en `base_config.h`.
   - Gira bien pero las rpm salen negativas → cambiar `ENC_INVERTIDO`.
4. **Cuentas por vuelta**: con la base liberada (`l`), marcar la rueda, girarla a mano 10 vueltas hacia adelante y leer `vueltas` en `e`. Si no da 10: nuevo valor = 8316 × 10/vueltas leídas.
5. `log 20` y `w 20 20 3`: CSV de la respuesta del lazo de velocidad, para graficar.
6. `v 0.1 0 2`, `v 0 0.5 2`: movimiento con rampa.
7. Probar la seta: apretarla → EMERGENCIA, 0 V en la bornera +12 V, `armar` y `v` rechazados. Soltarla: las ruedas **no** arrancan solas; hace falta `r`.
8. Probar el watchdog: `ros2 topic pub -r 10 /cmd_vel ...` y cortar con Ctrl+C: frena a los 300 ms.

## 13.4 Paso 3: base en el piso

1. `odo0`, `v 0.1 0 3` y medir con cinta cuánto avanzó. Comparar con $x$ en `e`: corregir `radioRueda` en `CFG_DIF`.

2. Girar 360° en el lugar (`v 0 0.5`) y comparar el ángulo: corregir la `trocha`.
3. `cal` con el robot quieto (sesgo del giróscopo).
4. Teleop con el teclado desde la Pi:

```
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 'armar'}"
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -p repeat_rate:=10.0
```

5. Medir la distancia de frenado del watchdog a 0,25 m/s.
6. Grabar un rosbag de `/cmd_vel` y `/odom` (criterio M1 de la arquitectura).

## 13.5 Paso 4: sensores en ROS

- `/range/frente`: acercar una caja; tiene que bajar. A menos de 0,20 m la base no avanza.
- `/bumpers`: apretar cada uno; bit 0 frente, 1 izquierda, 2 derecha.
- `/imu/data_raw`: inclinar la base; a más de 25° entra en VUELCO.
- `/battery`: comparar con el tester.
- `/gps/fix` (si el GPS está conectado a la Pi): `cat /dev/ttyAMA0` tiene que mostrar frases `$GP...`.

## 13.6 Paso 5: visión

1. `~/scripts/probar_camaras.sh`: las dos cámaras a 30 FPS.
2. `start_limon.sh` con el modelo (capítulo 9).
3. Limón real a 40/55/70/90 cm en el eje: `/limon/posicion` contra cinta.
4. Prueba de campo en un arbusto y al sol (NoIR).

## 13.7 Paso 6: brazo con J1

Una articulación a la vez, con la fuente limitada a 2–3 A si se puede.

1. Reflashear la WROOM con los pines v0.5.
2. `d 1 20` repetido: ver con `e` que la velocidad tenga el mismo signo que el duty.
3. `v 1 10 3`: lazo de velocidad solo.
4. Llevar J1 a la marca del banco (brazo hacia la izquierda) y `z 1`. O `home 1` con el final de carrera.
5. `j 1 120`, `j 1 60`: movimientos en posición.
6. Con micro-ROS: `armar` y `/brazo/joint_cmd`.

## 13.8 Paso 7: J2 y J3 (cuando lleguen los motores)

1. Montar con el brazo **apoyado**: no se sostienen solos.

2. Medir signos (`ENC_INVERTIDO`, `MOTOR_INVERTIDO`) con `d`.
3. Identificar zona muerta, $K$, $\tau$ y ajustar el PI (pasos 0–3 de `pruebas-esp32/10`).
4. Medir `HOME_Q` (ángulo de cada final de carrera) y `POSE_CERO`.
5. Ajustar `kpPos`, `velMax`, `accMax` con carga.
6. Probar `/brazo/objetivo` con `brazo_ik` a varios puntos medidos con cinta.

## 13.9 Paso 8: TF cámara → brazo

1. Con el soporte de cámaras montado en el mástil, medir con cinta $x$, $y$, $z$ de `stereo_camera_link` respecto de `brazo_base`.
2. Lanzar `mision.launch.py` con esos valores y ángulos aproximados.
3. Poner un limón en un lugar medido a mano y comparar con lo que da la TF (`ros2 run tf2_ros tf2_echo brazo_base camera` + `/limon/posicion`). Ajustar roll, pitch y yaw.
4. Medir `largo_herramienta`.

## 13.10 Paso 9: misión

1. Base quieta, limón a la vista y al alcance. Mano en la seta.
2. `cierre` bajo al principio (por ejemplo 40) y subirlo hasta que agarre sin aplastar.
3. `ros2 launch mision mision.launch.py ...` y seguir `/mision/estado`.
4. Video de al menos 5 cosechas seguidas (criterio M4).

## 13.11 Paso 10: recorrido autónomo

1. Robot en el centro del pasillo, mirando a lo largo.
2. Parámetros de la zona útil de la cámara real (`f_px`, `u_min`).
3. `autonomo:=true`, con alguien al lado con la mano en la seta.

CAPÍTULO 14

# Diagnóstico: qué hacer si algo no anda

## 14.1 Primero, mirar el estado

Casi todo problema aparece en el texto de estado de cada placa:

```
ros2 topic echo /base/estado --field data       # modo, seguridad, bloqueos, IMU, fallas, enlace
ros2 topic echo /brazo/estado --field data      # armado; por articulación: modo, cero, duty,
    falla, llego
ros2 topic echo /base/respuesta --field data    # respuesta a cada comando (por qué se rechazó)
ros2 topic echo /brazo/respuesta --field data
ros2 topic echo /mision/estado --field data     # estado y motivo de la misión
ros2 topic echo /brazo/ik_estado --field data   # por qué un punto es inalcanzable
```

## 14.2 Tabla de síntomas

| Síntoma | Causa probable | Qué hacer |
|---|---|---|
| **Enlace y micro-ROS** | | |
| `/odom` o `/brazo/joint_states` no aparecen | Agente no corriendo, puerto equivocado o placa sin hora | Arrancar el agente con el puerto de `/dev/serial/by-id/`; esperar unos segundos |
| La S3 no abre sesión | Quedó en modo descarga | `esptool -before no_reset -after watchdog-reset read-mac` |
| El brazo se reconecta cada ~1 s | Baudios altos en el CH340 | Usar 115 200 (el agente con `-b 115200` si se cambió) |
| `pings_fallidos` o `cortes` suben | USB flojo o la Pi sin energía | Revisar cables; `dmesg`; `vcgencmd get_throttled` |
| No se puede flashear | El agente tiene el puerto | Detener el agente antes de `pio run -t upload` |
| **Base** | | |
| `/cmd_vel` no mueve nada | Base desarmada | `armar` por `/base/comando` |
| Armada pero no se mueve | Seta, vuelco, IMU sin datos, sin eco al frente, obstáculo cerca | Leer `/base/estado`; despejar la causa y `r` |
| Avanza a los tirones y frena | `/cmd_vel` publicado con `-once` o teleop sin `repeat_rate` | Publicar a 10 Hz (`-r 10`; `repeat_rate:=10.0`) |
| Una rueda gira al revés | Signo del motor | `MOTOR_INVERTIDO` |
| rpm negativas yendo adelante | Signo del encoder | `ENC_INVERTIDO` |
| Avanza menos o más que lo pedido | Radio de rueda o cuentas por vuelta | Medir con cinta y corregir `CFG_DIF` |
| Gira de más o de menos | Trocha | Medir y corregir `trocha` |

| Síntoma | Causa probable | Qué hacer |
|---|---|---|
| Falla de rueda / atasco | Rueda trabada o encoder desconectado | Revisar; `r` |
| Encoder cuenta solo, quieto | Entrada al aire (falta pull-up) o ruido | Revisar cableado y pull-ups |
| `BATERIA_BAJA` con la batería llena | Seta apretada (divisor en 0 V) o divisor mal | Normal con la seta; si no, medir el divisor |
| Bumper o seta disparan solos | Ruido (no hay capacitores) | 100 nF de ese pin a GND |
| **Brazo** | | |
| Rechaza movimientos | Desarmado o sin cero | `armar`; `z` en la pose de referencia o `home <n>` |
| `FueraDeLimites` | Se movió a mano o el cero está mal | `r`; rehacer el cero |
| Atasco | Obstáculo, correa floja o encoder desconectado | Revisar; `r` |
| Tiembla cerca del objetivo | PI o $v_{\text{mín}}$ mal ajustados | Bajar $K_p$; ajustar `velMin` y tolerancia |
| J2 o J3 caen | No se autobloquean | Apoyar el brazo; nunca `l` con el brazo levantado |
| `inalcanzable` en `/brazo/ik_estado` | Punto fuera del espacio de trabajo o del rango de J1 | Leer el motivo; acercar la base |
| Servo da un salto al encender | GPIO 14 pulsa al arrancar | Conectar los 6 V después de que arranque la WROOM, o pasar la señal al GPIO 13 |
| **Visión y misión** | | |
| Sin `/limon/posicion` con el limón a la vista | Limón en la zona ciega, fuera de 0,38–0,96 m o robot en movimiento | Correr el robot; frenar |
| Posiciones que saltan | Exposición sin sincronizar, robot moviéndose | Localizar quieto |
| Cámara no arranca | Cinta CSI | Intercambiar cintas con la Pi apagada |
| Colores rosados al sol | Cámaras NoIR | Pendiente: probar o sombrear |
| Misión en FALLA | Ver el motivo en `/mision/estado` | Corregir y reiniciar el nodo |
| Misión se queda en ESPERAR | Limón inestable, brazo no quieto o fuera de alcance | Ver `/mision/estado`; acercar la base |
| La simulación mueve el robot real | Falta aislar | `ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST` |
| **Energía** | | |
| La Pi se reinicia o `get_throttled` $\neq$ `0x0` | Power bank al límite o cable largo | Una sola salida, cable corto; bajar carga |
| Salta el fusible | Cortocircuito | **No** reemplazar sin encontrar el corto |

## 14.3 Comandos útiles de la Pi

```
vcgencmd get_throttled                # 0x0 = sin problemas de alimentación ni temperatura
```

```
vcgencmd pmic_read_adc EXT5V_V          # tensión de entrada (>= 4,9 V)
dmesg -w                                # desconexiones USB
ls -l /dev/serial/by-id/                # qué placa está en qué puerto
htop                                    # uso de CPU (YOLO + SGBM son lo más pesado)
```

CAPÍTULO 15

# Estado del proyecto y lo que falta

## 15.1 Qué está hecho y qué no (14/10/2026)

| Parte | Estado | Detalle |
|---|---|---|
| Firmware de la base | ✓ código | 20 pruebas OK; micro-ROS probado con la placa sola (tasas nominales, armado, reconexión). **Sin probar con motores ni sensores.** |
| Firmware del brazo | ✓ código | 17 pruebas OK; J1 medido en el banco (`pruebas-esp32/10`). Pines v0.5 sin flashear en la Pi. J2/J3 sin motores. |
| `brazo_ik` | ✓ | 7 pruebas OK; probado en la notebook. |
| `mision` y `recorrido` | ✓ sim | 30 pruebas OK; 12 de 12 limones en la simulación. Sin probar con el brazo real. |
| Simulación | ✓ | Gazebo Harmonic con base, brazo, sensores y hilera. |
| Visión | ✓ lab | Estéreo calibrado (RMS 0,36 px); limón real con error 3D $\leq$ 17 mm. Falta la prueba de campo. |
| YOLO | parcial | Una clase (`limon`); B3 pide tres. |
| Potencia | armando | Diseño v0.7 revisado por Codex; faltan el portafusible y cambiar el Deans por XT60. |
| Placas | armando | Diseño de las perforadas y los shields listo (`docs/html/`). |
| Cableado general | ~10 % | |
| GPS | configurado | Pi lista, falta conectar el módulo. |
| App móvil, luz, buzzer | después | Después de la prueba integrada. |

## 15.2 Valores marcados MEDIR

| Valor | Dónde | Cómo |
|---|---|---|
| `cuentasPorVuelta` (8316) | `base_config.h` | 10 vueltas a mano (paso 2) |
| Radio de rueda (0,075) y trocha (0,40) | `CFG_DIF` | cinta, avance y giro de 360° (paso 3) |
| Signos de motores y encoders | `base_config.h`, `brazo_config.h` | `d` y `e` |
| Modelo y PI de las ruedas | `CFG_RUEDA` | `log` + `d` |
| J2/J3: $K$, $\tau$, zona muerta, PI | `CFG_ART` | pasos 0–3 de `pruebas-esp32/10` |
| `HOME_Q`, `POSE_CERO` | `brazo_config.h` | con la mecánica armada |
| Pulsos de los servos | `brazo_config.h` | sin forzar la pinza contra el tope |
| Balancín del cuatro barras (40 o 60 mm) | `cinematica.py` | calibre |
| `largo_herramienta` | `brazo_ik` y `mision` | muñeca → centro del agarre |
| `cierre` | `mision` | con un limón, sin aplastar |

| Valor | Dónde | Cómo |
|---|---|---|
| TF `brazo_base` → `stereo_camera_link` | `mision.launch.py` | cinta + limón medido |
| Zona útil de la cámara real | `ParRecorrido` | $f = 654{,}3$, $u_{\text{mín}} \approx 163$ |
| Parada real por watchdog | — | cinta, a 0,25 m/s |
| Peso del robot y autonomía real | — | balanza; pinza amperométrica |

## 15.3 Plan hasta la entrega (9/11/2026)

| Semana | Objetivo |
|---|---|
| 6–12/10 | Potencia + S3 + sensores + visión real |
| 13–19/10 | Base con teleop, GPS/IMU, soporte de cámaras, J1 por micro-ROS, IBT-2 de J2/J3 cableados |
| 20–26/10 | Hilera real, app, TF |
| 27/10–2/11 | J2/J3 (partiendo de J1) y `mision` real |
| 3–9/11 | Integración, 30 frutos, informes |

## 15.4 Mantener la Pi al día

La Pi tiene una copia del repo por `rsync` (sin `.git`) en `~/recolector_de_frutas`. El comando `/pi-sync` de Claude Code hace: `rsync` del repo, `colcon build` de los paquetes que cambiaron y, si cambió el firmware, compilar y flashear los ESP32 desde la Pi. Pendiente a la fecha: recompilar `mision` (seta) y reflashear la base (LiPo, 08/10) y el brazo (pines v0.5, 12/10).

| En la Pi | Qué hay |
|---|---|
| `~/recolector_de_frutas` | copia del repo |
| `~/vision_estereo` | repo de visión (clon de git) |
| `~/microros_ws` | agente micro-ROS |
| `~/firmware/<base\|brazo>` | binarios para flashear |
| `~/venvs/esptool` | esptool |
| `~/scripts`, `~/logs` | scripts y registros |
| `~/ros/`, `~/venvs/` | workspaces y entornos de Python |

CAPÍTULO A

# Referencia rápida

## A.1 Todos los tópicos

| Tópico | Tipo | Publica | Suscribe |
|---|---|---|---|
| /cmd_vel | geometry_msgs/Twist | recorrido, teleop | S3 |
| /odom | nav_msgs/Odometry | S3 (25 Hz) | recorrido |
| /imu/data_raw | sensor_msgs/Imu | S3 (50 Hz) | recorrido |
| /range/frente\|izq\|der | sensor_msgs/Range | S3 (10 Hz) | recorrido |
| /bumpers | std_msgs/UInt8 | S3 (10 Hz) | recorrido |
| /battery | sensor_msgs/BatteryState | S3 (1 Hz) | — |
| /base/estado | std_msgs/String | S3 (2 Hz) | mision, recorrido |
| /base/comando | std_msgs/String | recorrido, a mano | S3 |
| /base/respuesta | std_msgs/String | S3 | — |
| /gps/fix | sensor_msgs/NavSatFix | nmea_navsat_driver (Pi) | — |
| /brazo/joint_states | sensor_msgs/JointState | WROOM (20 Hz) | mision |
| /brazo/estado | std_msgs/String | WROOM (2 Hz) | mision |
| /brazo/comando | std_msgs/String | mision, a mano | WROOM |
| /brazo/respuesta | std_msgs/String | WROOM | — |
| /brazo/joint_cmd | sensor_msgs/JointState | brazo_ik | WROOM, mision |
| /brazo/objetivo | geometry_msgs/PointStamped | mision | brazo_ik |
| /brazo/ik_estado | std_msgs/String | brazo_ik | mision |
| /pinza/cierre | std_msgs/Float32 | mision | WROOM |
| /pinza/giro | std_msgs/Float32 | a mano | WROOM |
| /yolo/detecciones | vision_msgs/Detection2DArray | detector_node | localizador, recorrido |
| /limon/posicion | geometry_msgs/PointStamped | localizador_node | mision |
| /mision/estado | std_msgs/String | mision | — |
| /recorrido/estado | std_msgs/String | recorrido | — |

## A.2 Comandos de todos los días

```
# --- Conectarse y preparar
ssh r2d2@100.116.96.71
source /opt/ros/jazzy/setup.bash

# --- Agentes micro-ROS (una terminal cada uno)
source ~/microros_ws/install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/serial/by-id/usb-
    Espressif_USB_JTAG_serial_debug_unit_*-if00
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/serial/by-id/usb-1a86_USB_Serial-if00-
    port0

# --- Base
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 'armar'}"
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -p repeat_rate:=10.0
ros2 topic pub --once /base/comando std_msgs/msg/String "{data: 's'}"
```

```
# --- Brazo
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 'z'}"
ros2 topic pub --once /brazo/comando std_msgs/msg/String "{data: 'armar'}"
ros2 topic pub --once /brazo/joint_cmd sensor_msgs/msg/JointState "{name: [j1], position:
    [1.57]}"
ros2 topic pub --once /pinza/cierre std_msgs/msg/Float32 "{data: 50.0}"

# --- Visión
MODELO=~/vision_estereo/modelos/limon_yolo26n_ncnn_model CLASE=limon ~/vision_estereo/scripts/
    start_limon.sh

# --- Misión
ros2 launch mision mision.launch.py x:=... y:=... z:=... roll:=... pitch:=... yaw:=...

# --- Firmware (en la notebook)
cd firmware/base && ~/.platformio/penv/bin/pio test -e native && ~/.platformio/penv/bin/pio run -
    e ros

# --- Grabar para la entrega
ros2 bag record /cmd_vel /odom /imu/data_raw /range/frente /bumpers /base/estado /limon/posicion
    /mision/estado
```

## A.3 Estructura del repositorio

```
recolector_de_frutas/
|-- CLAUDE.md                 estado y reglas del proyecto
|-- docs/
|   |-- INDICE.md             índice de documentos
|   |-- ARQUITECTURA.md       arquitectura del objetivo mínimo
|   |-- CONEXIONES.md         pines, potencia, autonomía, "antes de conectar"
|   |-- BITACORA.md           bitácora por fecha
|   |-- html/                 páginas interactivas (todo.html junta todas)
|   |-- manual/               este manual (manual.tex)
|   |-- fotos/                hojas de datos y fotos
|   |-- TPF R1/, TPF R2/      entregas
|-- firmware/
|   |-- lib/                  drivers y control compartidos
|   |-- base/                 ESP32-S3 (PlatformIO)
|   `-- brazo/                ESP32-WROOM (PlatformIO)
|-- ros2_ws/src/
|   |-- brazo_ik/             cinemática inversa
|   |-- mision/               misión + recorrido + TF cámara → brazo
|   `-- simulacion/           Gazebo Harmonic
`-- vision/entrenamiento_limon/   cuaderno de Colab del detector
```

## A.4 Documentos relacionados

- `docs/CONEXIONES.md`: la fuente de los pines, la potencia y la lista "Antes de conectar".
- `docs/html/todo.html`: todas las páginas de cableado en una, con cajas clickeables.
- `firmware/base/README.md`, `firmware/brazo/README.md`: detalle del firmware.
- `ros2_ws/src/*/README.md`: detalle de cada paquete.
- `~/vision_estereo/README.md`: detalle de la visión.
- `docs/BITACORA.md`: por qué se tomó cada decisión.